

Claude Code Deep Dive Workshop

Chapter 1 - Overview

Agentic Coding의 새로운 표준, Claude Code 완전 정복

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com



About this Workshop

3-Day Deep Dive into Claude Code

WORKSHOP MISSION

본 워크샵은 Claude Code를 실무에 즉시 적용 가능한 수준까지 학습합니다.

단순 사용법을 넘어 아키텍처, 보안, 자동화, SDK 통합까지 다룹니다.

DAY 1

Foundations

Overview, Subagents의 개념과 기본 사용법을 학습합니다. 6시간 강의 + 2시간 실습.

DAY 2

Enterprise

Admin Setup, Settings로 대규모 조직 배포와 정책 구성을 학습합니다. 6시간 + 2시간.

DAY 3

Automation

CLI Reference, Agent SDK로 자동화/통합/맞춤 에이전트 개발을 학습합니다. 8시간.

OUTCOME

Production-Ready

워크샵 후 사내 환경에 Claude Code를 배포하고 자동화 파이프라인까지 구축할 수 있습니다.

3

Days of intensive training

6

Major chapters

1,200+

Slides of content

15+

Hands-on labs

Agenda

Claude Code Workshop

01

Overview

Claude Code 정체성, 아키텍처, 설치, 핵심 기능

02

Agents (Subagents)

맞춤 서브에이전트 설계와 병렬 작업 위임

03

Admin Setup

Enterprise 배포, 자격증명, 사내망 통합

04

Settings

settings.json, Hooks, MCP 서버 통합

05

CLI Reference

전체 CLI 플래그, Headless, 자동화

06

Agent SDK

Python/TypeScript SDK 맞춤 개발

Lab Environment

사전 준비사항 및 실행 환경

PREREQUISITES

아래 환경이 준비되어 있어야 모든 실습을 원활히 진행할 수 있습니다. 설치하는 챕터 1 Part 3에서 함께 진행합니다.

OS

macOS / Linux / WSL2

macOS 12+, Ubuntu 20.04+,
Windows 11 with WSL2 권장.
Native Windows도 지원되나 일
부 도구 제약 존재.

RUNTIME

Node.js 18 LTS+

Node.js 18 LTS 이상 필수. nvm
또는 fnm으로 버전 관리 권장.
Native installer 사용 시 Node
불필요.

ACCOUNT

Anthropic / AWS / GCP

Anthropic 계정 또는 AWS
Bedrock 액세스 필요.
실습 중 임시 자격증명을 제공.

TOOLS

Git, IDE, Terminal

Git, VS Code 또는 JetBrains
IDE, 그리고 익숙한 터미널 예물
레이터를 준비.

8GB+

RAM 권장

10GB+

Free disk space

HTTPS

api.anthropic.com 접근

443

Outbound port 필요

Learning Outcomes

본 워크샵 종료 시 달성할 역량

BY THE END YOU WILL

Claude Code를 사용 가능한 수준이 아니라, 운영 가능한 수준까지 학습합니다. 즉, 도입, 보안, 운영, 자동화의 4축을 모두 다룹니다.

OUTCOME 1 Hands-on Mastery

디버깅, 리팩토링, 신기능 개발 등 일상 워크플로를 Claude Code로 실행 가능.

OUTCOME 2 Customization

Subagents, CLAUDE.md, Hooks, MCP를 활용해 팀/프로젝트에 맞춰 동작을 조정 가능.

OUTCOME 3 Enterprise Ops

IAM 정책, 모델 라우팅, 사용량 모니터링을 통해 안전하게 조직 단위로 배포 가능.

OUTCOME 4 Automation

Headless 모드, Agent SDK를 활용해 CI/CD에 통합하고 자동화 파이프라인을 구축 가능.

4 Pillars

Mastery / Custom / Ops / Auto

15+ Labs

실습 중심 학습

Code

실제 코드와 명령 사용

Q&A

챕터별 토론 시간

CHAPTER 01

Overview

Claude Code의 정체성, 아키텍처, 설치, 인증, 핵심 기능, 워크플로 패턴까지



CHAPTER 01- PART 01

Overview – Claude Code 소개

Claude Code의 정체성, 아키텍처, 설치, 인증, 핵심 기능, 워크플로 패턴까지



Chapter 1 Learning Objectives

본 챕터에서 달성할 학습 목표

AT THE END OF THIS CHAPTER

Claude Code가 무엇이고 왜 다른지 설명할 수 있으며, 본인 환경에 설치/인증을 마치고 첫 세션을 운영할 수 있습니다.

OBJ 1

Define & Differentiate

Claude Code의 정의와
Agentic Coding 개념을 설명하고, Copilot/Cursor와의 차이를 비교.

OBJ 2

Install & Authenticate

npm 또는 native installer
로 설치하고, 4가지 인증 방법 중 환경에 맞는 것을 선택.

OBJ 3

Use Core Tools

Read, Edit, Bash, Grep
등 핵심 도구를 사용해 일상
코딩 작업을 수행.

OBJ 4

Manage Context

CLAUDE.md, /clear,
/compact, Plan 모드로 세션과 컨텍스트를 효과적으로 관리.

10 Parts

구조화된 학습 경로

5 Labs

Chapter 1 실습 랩

~3.5h

강의 시간

~1.5h

실습 시간

Chapter 1 Detailed Agenda

10 Parts / 200 Slides

PART 01

What is Claude
Code

22p

PART 02

Architecture

18p

PART 03

Installation

22p

PART 04

Authentication

18p

PART 05

Quick Start

22p

PART 06

Core
Capabilities

28p

PART 07

IDE Integration

14p

PART 08

CLAUDE.md &
Memory

16p

PART 09

Workflow
Patterns

18p

PART 10

Recap & Labs

14p

10

Logical parts

200

Total slides

5

Embedded labs

Q&A

챕터 종료 후

Part 1 / What is Claude Code

정의, 차별점, 사용 사례를 통한 첫 이해

PART OBJECTIVE

Claude Code의 정체성을 명확히 이해하고, 기존 코딩 도구와의 차이점을 비교하며, 어떤 상황에서 가장 큰 가치를 발휘하는지 학습합니다.

TOPIC 1

정의와 배경

Claude Code의 공식 정의,
Anthropic 회사 배경,
Claude 모델 패밀리 소개

TOPIC 2

Agentic Coding

Code Completion vs
Agentic Coding의 차이,
자율성 스펙트럼 이해

TOPIC 3

도구 비교

Copilot, Cursor, Cline과
의 비교를 통한 Claude
Code의 포지셔닝

TOPIC 4

6대 사용 사례

디버깅, 리팩토링, 신기능,
학습, 테스트, DevOps 6가
지 시나리오

22 Slides

Part 1 분량

~40 min

예상 강의 시간

6 Cases

사용 사례 분석

3 Tools

경쟁 도구 비교

Claude Code 공식 정의

Anthropic이 만든 터미널 기반 Agentic Coding 도구

DEFINITION

Claude Code는 Anthropic이 개발한 명령행 기반의 Agentic Coding 도구입니다.

개발자의 자연어 지시를 받아 코드베이스를 이해하고 파일을 수정하며 명령을 실행해 작업을 완료합니다.

WHAT

Agentic Tool

자율적으로 도구를 호출하며 멀티스텝 작업을 완수하는 에이전트형 도구.

WHERE

In Your Terminal

별도 IDE 없이 친숙한 터미널 환경에서 동작하며 모든 IDE와 호환.

WHO BY

By Anthropic

Claude 모델을 만든 Anthropic이 직접 만든 1차 도구로 모델과의 통합이 가장 깊음.

CLI

명령행 기반

Sonnet/Opus

주력 모델

2024

최초 출시

GA

현재 정식 출시 상태

Anthropic 회사 배경

AI Safety를 핵심 가치로 둔 Frontier AI 연구소

COMPANY

Anthropic은 2021년 설립된 AI Safety 연구 기업으로, Claude 모델 패밀리를 개발.
OpenAI 출신 연구진이 안전한 AI 시스템 연구를 목적으로 설립.

FOUNDED 2021

Dario Amodei와 Daniela Amodei가 공동 설립.
OpenAI 출신 연구진 다수가 합류.

MISSION AI Safety

Helpful, Harmless,
Honest를 핵심 가치로 두는
Constitutional AI 접근법을 개발.

PRODUCT Claude

Claude 3, 3.5, 4, 4.5 등
지속적으로 향상되는 거대
언어모델 시리즈를 제공.

PARTNERS AWS, Google

AWS Bedrock이 1급 파트너로 통합.

San Francisco
본사 위치

1,500+
직원 수 (2025)

Series F
최근 투자 라운드

Amazon Bedrock
클라우드 파트너

Claude 모델 패밀리

Claude Code가 활용하는 모델 라인업

Model	특성	권장 용도
Claude Opus	최고 성능 모델, 깊은 추론, 복잡한 코드 분석	대규모 리팩토링
Claude Sonnet	균형 잡힌 성능과 속도, Claude Code 기본 모델	일상 코딩 작업
Claude Haiku	빠른 응답, 경량 작업, 비용 효율적	간단한 변환/검색

AUTO ROUTING

Claude Code는 작업 복잡도에 따라 모델을 자동 선택할 수 있으며, `/model` 명령 또는 `--model` 플래그로 명시적 지정도 가능합니다.

200K
컨텍스트 윈도우 (기본)

Sonnet
Claude Code 기본

Opus
플랜 모드 권장

4.5
현재 최신 버전

Agentic AI 개념

Reactive에서 Agentic으로의 패러다임 전환

DEFINITION

Agentic AI는 사용자의 목표를 받아 스스로 계획을 세우고, 도구를 사용하며, 결과를 관찰하면서 목표 달성까지 자율적으로 작동하는 AI 시스템.

PROPERTY 1

Goal-driven

단계별 명령이 아닌 최종 목표를 받아 스스로 경로를 결정.

PROPERTY 2

Tool-using

파일 읽기, 명령 실행, 웹 검색 등 외부 도구를 능동적으로 호출.

PROPERTY 3

Observative

도구 호출 결과를 관찰하고 다음 행동을 결정하는 피드백 루프를 가짐.

PROPERTY 4

Iterative

실패하면 다른 접근법을 시도하며 목표 달성까지 반복.

ReAct

대표 패턴 (Reason + Act)

Plan→Act

실행 흐름

Multi-step

여러 단계 자동 실행

Self-correct

오류 시 자가 수정

Agentic Coding vs Code Completion

두 패러다임의 결정적 차이

CODE COMPLETION

제안 기반 보조 (Suggest)

1. 다음 줄/블록 자동 완성
2. 사용자가 모든 결정을 내림
3. 한 번에 한 파일만 다룸
4. 외부 도구 사용 없음
5. 컨텍스트는 현재 파일 중심

AGENTIC CODING

목표 지향 실행 (Execute)

1. 멀티스텝 작업을 자율 실행
2. 도구를 능동적으로 호출
3. 여러 파일을 동시에 수정
4. 명령 실행/검색/웹 접근 가능
5. 프로젝트 전체를 컨텍스트로 활용

Suggest

보조 도구의 동작

Execute

Claude Code의 동작

1 file

Completion의 작업 범위

Repo-wide

Agentic의 작업 범위

주요 코딩 AI 도구 비교

Claude Code 포지셔닝 이해

Tool	주요 특성	비교 우위 영역
GitHub Copilot	IDE 내장 자동 완성, Microsoft 통합 강점	실시간 코드 제안
Cursor	AI 우선 설계 IDE, 자체 에디터 기반	풀스택 IDE 통합
Cline / Roo	VS Code 확장형 에이전트, 멀티 모델 지원	IDE 확장 친화
Claude Code	터미널/Headless 우선, Anthropic 1차 통합, SDK	자동화/CI/CD 통합

POSITIONING

Claude Code는 IDE 의존성을 최소화하고 헤드리스 자동화, 자체 SDK 통합, 엔터프라이즈 IAM에 강점을 가집니다.

CLI-first

Claude Code 특징

IDE-first

Cursor 특징

Completion

Copilot 영역

Agentic

Claude Code 영역

왜 터미널 기반인가

CLI-first 설계의 4가지 합리적 이유

REASON 1

IDE 독립성

IDE에 묶이지 않으면 모든 에디터, 원격 SSH, 컨테이너 환경에서 동일하게 동작.

REASON 2

자동화 친화

셸 파이프, cron, CI/CD 등 모든 자동화 인프라와 자연스럽게 통합.

REASON 3

서버 운영 적합

GUI가 없는 서버, 원격 머신에서도 그대로 사용할 수 있어 DevOps 친화적.

REASON 4

경량/단순

복잡한 IDE 상태 동기화 대신 stdin/stdout 같은 간단한 인터페이스로 설계.

INSIGHT

터미널 기반이라고 IDE를 못 쓰는 것이 아닙니다. VS Code/JetBrains 플러그인이 따로 제공되어 양쪽 모두 활용할 수 있음.

All IDEs

동시 호환

SSH

원격 환경 가능

Docker

컨테이너 호환

CI/CD

파이프라인 통합

Claude Code의 4대 핵심 가치

Why teams adopt Claude Code

VALUE 1

깊은 코드 이해

단일 파일이 아닌 프로젝트 전체를 자율적으로 탐색하며 컨텍스트를 구축.

VALUE 2

실제 작업 완수

제안에 그치지 않고 파일 수정, 테스트 실행, PR 생성까지 직접 수행.

VALUE 3

안전한 권한 모델

allowedTools, ask, deniedTools로 동작 범위를 사전 통제하고 사용자가 승인.

VALUE 4

확장 가능한 통합

Subagents, MCP, Hooks, SDK를 통해 조직 워크플로에 맞춰 확장.

WHY IT MATTERS

이 4가지 가치는 개인 생산성 향상을 넘어, 팀과 조직 단위로 코딩 방식을 재정의할 수 있는 기반을 제공.

Repo-wide

컨텍스트 범위

End-to-end

작업 완수 수준

Granular

권한 제어

SDK + MCP

확장성

Read / Edit / Run

Claude Code의 3원 능력

READ

읽기 능력

소스 파일, 이미지, PDF, 디렉토리 구조
까지 자율적으로 읽고 이해.
Grep과 Glob으로 빠르게 검색.

EDIT

편집 능력

Find&Replace 단위 정밀 편집, 신규 파
일 생성, 멀티 파일 동시 수정 모두 가능.
Diff를 검토.

RUN

실행 능력

셸 명령, 테스트, 빌드, Git 명령을 직접
실행하고 결과를 관찰하여 다음 행동을
결정.

WHY 3 ABILITIES MATTER

읽기/편집/실행 3개가 모두 갖춰져야 진정한 End-to-end 코딩 에이전트가 되며, 하나만 빠져도 사람의 개입이 필요.

Read

파일/이미지/PDF

Edit

Find&Replace 정밀 편집

Run

Bash/Test/Git 실행

Loop

Run 결과로 다음 단계

자율성 스펙트럼

Suggest → Approve → Auto

STEP 01

Suggest
도구만 제안

STEP 02

Approve
단계별 승인

STEP 03

Plan Mode
사전 계획 후 일괄 승인

STEP 04

Auto-accept
위험 외 자동 진행

STEP 05

Yolo
전체 자동 (위험)

CONTROL DIAL

1. 왼쪽에서 오른쪽으로 갈수록 자율성이 증가하고 개입은 감소.
2. 환경의 위험도와 작업 성격에 맞춰 선택.
3. CI/CD에서는 보통 Headless + 사전 합의된 권한으로 운영.

Shift+Tab

Plan 모드 토글

auto-accept

편집 자동 수락

--dangerously

Yolo 활성화 (위험)

Default

Approve 모드

주요 사용 사례 6선

Claude Code가 가장 큰 가치를 발휘하는 시나리오

CASE 1

디버깅

에러 메시지에서부터 원인을 추적하고 수정안을 제시/적용

CASE 2

리팩토링

대규모 코드베이스의 일관된 패턴 변경/구조 개선

CASE 3

신기능 개발

요구사항으로부터 계획, 구현, 테스트까지 일괄 처리

CASE 4

코드베이스 학습

낯선 프로젝트의 구조와 흐름을 자연어 대화로 학습

CASE 5

테스트 작성

커버리지 갭 분석과 자동 테스트 생성

CASE 6

DevOps 자동화

셸 스크립트, IaC, CI/CD 파이프라인 작성과 점검

Daily

일상 코딩 적용

Scale

리팩토링/마이그레이션

Quality

테스트/리뷰

Ops

DevOps/자동화

Case 1 / 디버깅

에러 메시지로 시작하는 자율 추적

TYPICAL FLOW

User: 다음 에러가 발생합니다. 원인을 찾고 수정해 주세요.

```
TypeError: Cannot read property 'id' of undefined
    at UserService.findById (src/services/user.js:42)
    at AuthController.login (src/controllers/auth.js:78)
```

Claude:

1. Read src/services/user.js (line 30-60)
2. Read src/controllers/auth.js (line 60-90)
3. Grep "findById" → 3 callsites
4. Edit src/services/user.js (null check 추가)
5. Bash npm test -- user.test.js

KEY INSIGHT

에러 메시지에서 출발해 관련 파일을 자율 탐색하고, 수정 후 테스트 실행까지 한 번에 처리.

Read

관련 파일 자동 식별

Grep

호출 지점 전체 분석

Edit

안전한 수정 적용

Bash

테스트로 즉시 검증

Case 2 / 리팩토링

코드베이스 전반의 일관된 패턴 변경

User: 모든 axios 호출을 신규 ApiClient 래퍼로 마이그레이션해 주세요.

TYPICAL FLOW

Claude:

1. Grep "axios\.(get|post|put|delete)" → 47 occurrences
2. Read 각 파일의 호출 패턴 분석
3. Plan 공통 변환 규칙 도출
4. Edit 47곳 일괄 변환 (Find&Replace + 문맥 검토)
5. Bash npm run lint && npm test
6. Bash git diff --stat → 변경 요약 보고

KEY INSIGHT

Grep으로 영향 범위 식별, Plan으로 규칙 합의, 일괄 Edit, 자동 검증의 4단계가 대규모 리팩토링의 핵심 패턴.

47

변경 대상 수

Plan First

사전 계획 권장

Diff

PR 검토 가능

Test

회귀 자동 점검

Case 3 / 신기능 개발

요구사항부터 PR 생성까지

STEP 01

요구사항
파싱

STEP 02

코드베이스
탐색

STEP 03

설계 Plan
사용자 승인

STEP 04

구현 +
테스트

STEP 05

Git Commit
+ PR 생성

User: /users API에 페이지네이션을 추가하고 테스트도 작성해 주세요.

TYPICAL FLOW

Claude:

1. Read src/routes/users.js, src/services/user.js, tests/
2. Plan page/limit 쿼리, links 헤더, 기본 20, 최대 100
→ 사용자에게 계획 제시
3. Edit routes, service, repository 3개 파일 수정
4. Write tests/users.pagination.test.js 신규 작성
5. Bash npm test → green
6. Bash git checkout -b feat/users-pagination
7. Bash git commit / gh pr create

Plan

사용자와 합의 후 시작

Multi-file

여러 파일 동시 작업

Test

신규 테스트도 생성

PR

검토용 PR 자동 생성

Case 4 / 코드베이스 학습

낮선 프로젝트를 자연어 대화로 익히기

User: 이 프로젝트의 인증 흐름을 설명해 주세요.

TYPICAL FLOW

Claude:

1. Read package.json, README.md (스택 파악)
2. Glob "**/auth/**" → 관련 파일 12개 식별
3. Read src/auth/middleware.js, src/auth/jwt.js, ...
4. Grep "verify|authenticate|sign" 패턴 확인
5. Output Markdown 다이어그램과 함께 흐름 설명
 - Login → JWT 발급 → 토큰 검증 미들웨어
 - Refresh 토큰 회전 정책 / 권한 매트릭스

KEY INSIGHT

신규 합류 멤버의 온보딩을 며칠에서 몇 시간으로 단축할 수 있는 강력한 패턴입니다.

Onboarding

신규 멤버 온보딩 단축

Glob

관련 파일 자동 식별

Diagram

흐름 다이어그램 생성

CLAUDE.md

학습 내용 영구 저장

Case 5 / 테스트 작성

커버리지 갭 분석과 자동 생성

User: PaymentService의 테스트 커버리지를 90%로 끌어올려 주세요.

TYPICAL FLOW

Claude:

1. Bash `npm test -- --coverage` → 현재 67% 확인
2. Read `coverage/coverage-summary.json`
3. Grep 누락된 라인의 함수 식별 (refund, retry 등)
4. Read `src/services/payment.js` 미커버 분기 분석
5. Write `tests/payment.refund.test.js`
`tests/payment.retry.test.js`
6. Bash `npm test -- --coverage` → 92% 달성
7. Output 변경 요약 + 추가 권장 케이스

KEY INSIGHT

커버리지 메트릭과 코드 구조를 함께 보고 의미 있는 테스트를 생성. 분기/예외까지 다룸.

67% → 92%

커버리지 향상 예시

Branch

분기 커버리지 분석

Edge cases

예외 케이스 식별

Multi-test

테스트 파일 분리

Case 6 / DevOps 자동화

인프라/스크립트/파이프라인 작성

BASH

셸 스크립트

복잡한 ETL 스크립트나 백업 자동화를 자연어로 명세하면 안전한 셸 스크립트로 변환.

IAC

Terraform/CDK

AWS/GCP 리소스 IaC 코드를 사전 검토와 함께 생성/수정. terraform plan 자동 실행도 가능.

CI

GitHub Actions

워크플로 yaml을 분석하고 작성. matrix 빌드, 캐시 설정, 시크릿 처리도 다룸.

K8S

Kubernetes

manifest, Helm chart, kustomize 패치를 작성하고 kubectl로 검증까지 수행.

KEY INSIGHT

Claude Code는 일반 코딩만이 아니라 인프라 자동화 영역에서도 강력하게 작동하며, --dangerously-skip-permissions를 피하면서도 충분한 자동화를 달성.

Shell

셸 스크립트 생성

IaC

Terraform/CDK

CI/CD

GitHub Actions

K8s

Kubernetes manifests

라이선스 및 구독 모델

개인부터 엔터프라이즈까지

Plan	주요 특성	권장 대상
Free	API Key 기반, 종량제 비용	평가/소규모
Claude Pro	월 정액, 개인 일상 사용 분량	개인 개발자
Claude Max	더 큰 사용량, 우선 액세스	파워 유저
Team / Enterp.	SSO, 사용량 관리, 정책 통제, 청구 분리	조직 단위 도입

NOTE

구독제 외에도 AWS Bedrock을 통해 자체 클라우드 청구로 이용 가능. (기업들은 보통 이 경로를 선호).

API Key

소규모/평가

Pro/Max

개인 정액

Enterprise

SSO + 정책

Bedrock

클라우드 청구

데이터 보안 및 프라이버시

Anthropic 데이터 사용 정책

PRIVACY

학습 미사용

API 및 Claude Code를 통한 입력은 기본적으로 모델 학습에 사용되지 않음.

RETENTION

제한적 보존

안전성 점검 목적의 짧은 보존 기간을 두며, ZDR 옵션도 제공.

TRANSIT

TLS 1.2+

모든 API 통신은 TLS로 암호화되며 인증서 검증이 강제.

COMPLIANCE

SOC2 / HIPAA

SOC2 Type II 인증, BAA 통한 HIPAA 지원, 추가적 엔터프라이즈 컴플라이언스 항목 지원.

IMPORTANT

데이터 사용 정책은 플랜과 경로에 따라 다를 수 있으며, Amazon Bedrock 경로에서는 해당 클라우드의 데이터 처리 약관이 적용.

No-train

기본 비학습

ZDR

Zero Data Retention

SOC2

Type II 인증

HIPAA

BAA 가능

Anthropic 데이터 사용 정책 핵심

한 페이지 요약

항목	기본 정책	비고
학습 사용	미사용 (Opt-out 기본)	사용자 명시 동의 시만 사용
보존 기간	안전성 검토 30일 이내	ZDR 시 0일
인간 검토	학대 의심/안전 위반 시	자동화 시스템 우선
제3자 공유	없음 (법적 의무 외)	소환장 등 법적 요구 외
암호화	TLS in transit, AES-256 at rest	모든 경로 동일

Opt-out

학습 사용 거부 기본

30일

안전성 검토 최대 보존

ZDR

Zero Data Retention 옵션

AES-256

저장 데이터 암호화

Part 1 Recap

핵심 정리와 다음 파트 예고

TAKEAWAY 1

Agentic Coding

단순 자동 완성이 아닌, 목표를 받아 자율 실행하는 패러다임.

TAKEAWAY 2

Read/Edit/Run

3원 능력이 모두 갖춰져 End-to-end로 작업을 완수.

TAKEAWAY 3

자율성 다이얼

Suggest → Approve → Auto → Yolo로 환경에 맞춰 조절 가능.

TAKEAWAY 4

6가지 핵심 사례

디버깅, 리팩토링, 신기능, 학습, 테스트, DevOps 영역에서 가치 발휘.

NEXT - PART 2

Part 2에서는 Claude Code의 내부 아키텍처를 들여다봅니다.

CLI/IDE/SDK 계층, 모델 라우팅, 도구 시스템, 보안 아키텍처를 18장에 걸쳐 다룹니다.

Part 1

완료

Part 2

Architecture (18p)

Part 3

Installation (22p)

~3.5h

Chapter 1 잔여 시간

CHAPTER 01- PART 02

Overview - Architecture

Claude Code의 정체성, 아키텍처, 설치, 인증, 핵심 기능, 워크플로 패턴까지



Part 2 / Architecture

Claude Code 내부 구조 들여다보기

PART OBJECTIVE

Claude Code가 어떤 컴포넌트들로 구성되어 있는지, 사용자의 명령이 어떻게 처리되어 모델로 전달되고 도구가 실행되는지를 학습.

LAYER 1

클라이언트

CLI, IDE 플러그인, Agent SDK 세 가지 진입점을 모두 다룹니다.

LAYER 2

코어 & 모델

API 통신, 모델 라우팅, 컨텍스트 윈도우 관리의 내부 동작을 살펴봅니다.

LAYER 3

도구 시스템

내장 도구, MCP 통합, 권한 모델이 어떻게 협력하는지 학습합니다.

LAYER 4

보안 & 운영

데이터 흐름, 네트워크 트래픽, 텔레메트리 등 운영 관점을 다룹니다.

18 Slides

Part 2 분량

~30 min

예상 강의 시간

4 Layers

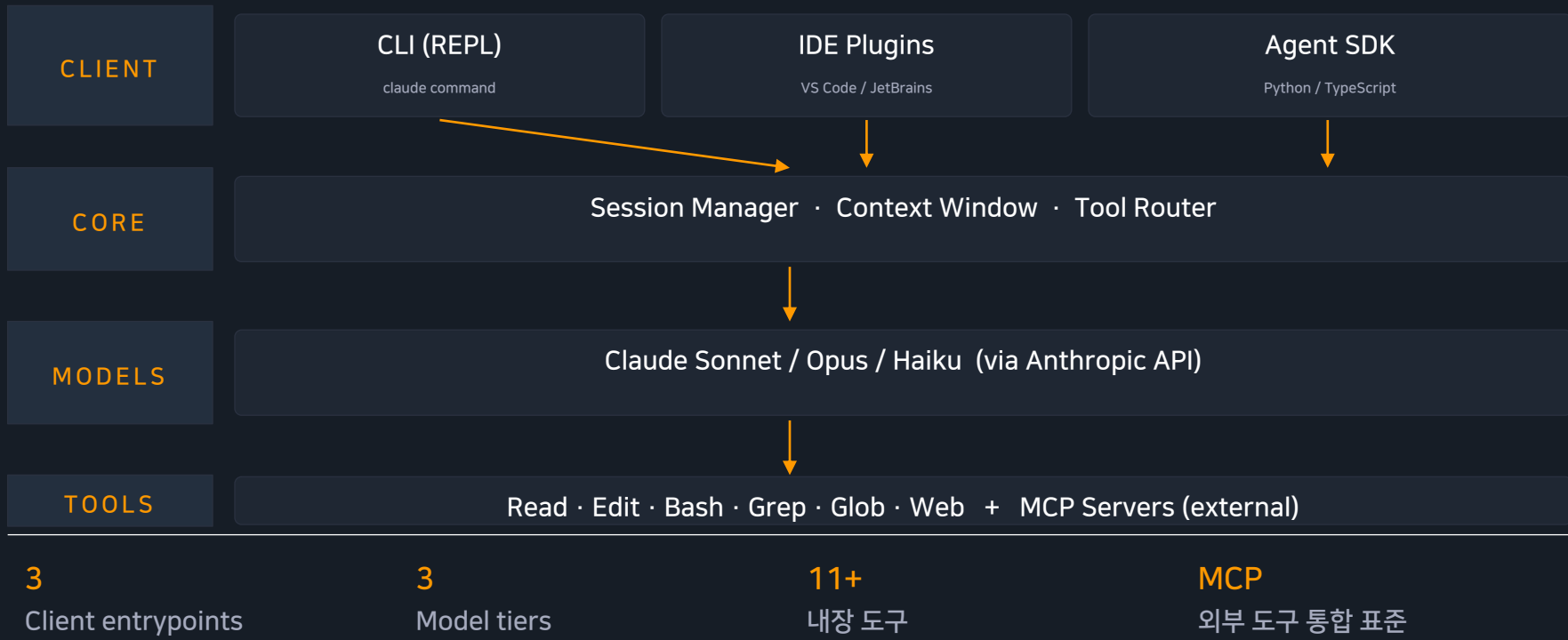
논리 계층

Deep Dive

내부 동작 중심

전체 아키텍처

Claude Code 시스템 다이어그램



Client Layer / CLI

터미널에서 동작하는 1차 인터페이스

CLI ENTRYPOINT

claude 명령어가 1차 진입점. 대화형 REPL과 헤드리스 모드 두 가지 실행 모드를 지원.

INTERACTIVE

대화형 REPL 모드

1. claude 명령으로 시작
2. 명령어 입력, 결과 출력 반복
3. 슬래시 명령 (/help, /clear) 지원
4. Plan/Auto/Approve 모드 토글
5. 다중 세션 관리 가능

HEADLESS

비대화형 모드

1. claude -p "prompt" 형식
2. stdin/stdout 파이프 지원
3. CI/CD 통합에 최적
4. JSON 출력 옵션
5. 자동화 스크립트 친화

claude

기본 진입점 cmd

-p

헤드리스 플래그

/

슬래시 명령 prefix

JSON

구조화 출력

Client Layer / IDE Plugins

VS Code와 JetBrains 통합

VS CODE

Claude Code Extension

Marketplace에서 설치.
사이드바 패널, 인라인 Diff, 컨텍스트
메뉴 통합을 제공.

JETBRAINS

IntelliJ Platform

IntelliJ, PyCharm, WebStorm 등 모
든 JetBrains IDE에서 동일한 플러그인
으로 사용 가능.

TERMINAL

Embedded Terminal

IDE 내장 터미널에서 claude를 실행하면
IDE 컨텍스트(선택 영역 등)와 연동.

INTEGRATION POINTS

선택 영역 자동 전달, Diff 인라인 표시, 파일 트리 동기화, 키보드 단축키 통합이 핵심 통합 지점.

VS Code

Marketplace 게시

JetBrains

플러그인 저장소

CLI

내장 터미널 연동

Shortcuts

키보드 단축키

Client Layer / Agent SDK

Python과 TypeScript로 프로그래매틱 통합

PYTHON SDK

```
# Python SDK 예시
from claude_agent_sdk import ClaudeAgent

agent = ClaudeAgent(
    cwd="/path/to/repo",
    allowed_tools=["Read", "Edit", "Bash"],
    model="claude-sonnet-4-5",
)

result = agent.run(
    "사용자 모델에 deleted_at 컬럼을 추가하고 마이그레이션을 생성해 주세요."
)

print(result.summary)
print(result.modified_files)
```

Python

주력 SDK 언어

TypeScript

병행 지원

Async

비동기 호출

Stream

스트리밍 응답

API Communication

클라이언트와 Anthropic API 사이의 통신

경로	엔드포인트	용도
Anthropic Direct	api.anthropic.com	API Key 또는 OAuth 인증
AWS Bedrock	bedrock-runtime.{region}.amazonaws.com	IAM 자격증명
Google Vertex AI	{region}-aiplatform.googleapis.com	ADC 자격증명
MCP Server	localhost 또는 사용자 지정 URL	외부 도구 통합

PROTOCOL

모든 API 통신은 HTTPS(TLS 1.2+)이며, SSE(Server-Sent Events) 기반 스트리밍을 통해 토큰 단위로 응답 수신.

HTTPS
TLS 1.2+

SSE
스트리밍 프로토콜

JSON
요청/응답 포맷

OAuth/Key
인증 방식

인증 계층

4가지 자격증명 흐름

METHOD 1

OAuth (Pro/Max)

브라우저 OAuth로 Claude
구독 계정 연동.
/login 명령으로 시작.

METHOD 2

Anthropic API Key

ANTHROPIC_API_KEY 환
경변수 또는
~/.claude/keys 파일.

METHOD 3

AWS Bedrock

AWS_REGION,
AWS_PROFILE 또는 IAM
Role 기반 자격증명 자동 사용.

METHOD 4

Vertex AI

GOOGLE_APPLICATION_
CREDENTIALS 또는
gcloud ADC 사용.

PRECEDENCE

CLI 플래그, 환경변수, 프로필 파일, 기본값 순으로 우선순위가 적용. (Part 4에서 상세히 다룹니다.)

4

Auth methods

Env

환경변수 우선

Role

IAM Role 자동 감지

ADC

GCP 기본 자격증명

Model Routing

Sonnet, Opus, Haiku의 자동 라우팅

Scenario	자동 선택 모델	이유
일반 코딩 (기본)	Sonnet 4.5, 4.6	균형 잡힌 성능과 속도
Plan 모드 추론	Sonnet 4.5, 4.6 (또는 Opus)	복잡한 계획 수립
Subagent (delegate)	Sonnet 4.5, 4.6 또는 Haiku	병렬 처리/비용 효율
명시적 지정	/model 명령 또는 --model 플래그	사용자 의도 우선

WHY ROUTING

작업 복잡도에 맞는 모델을 자동으로 선택해 비용과 응답 속도를 최적화합니다. 사용자는 필요 시 언제든지 오버라이드할 수 있습니다.

Sonnet

기본 코딩 모델

Opus

심층 추론용

Haiku

경량/빠름

/model

런타임 전환

Tool System 개요

도구 카탈로그와 호출 흐름

Tool Call Flow

1. Claude 응답 → tool_use 블록 (name, input)
2. Tool Router → 도구 식별 → 권한 검사
3. Permission Layer → allow / ask / deny 결정
4. Tool Executor → 실제 도구 실행 (Read/Edit/Bash/...)
5. 결과 → tool_result → 다음 Claude 호출의 컨텍스트로

Built-in Tools

Read · Edit · Write · NotebookEdit · Bash · Grep · Glob ·
WebSearch · WebFetch · TodoWrite · Task (subagent dispatch)

MCP Tools

외부 서버가 제공하는 도구를 동일한 인터페이스로 호출

TOOL CALL CYCLE

11+

내장 도구

MCP

외부 도구 표준

JSON

도구 입출력 포맷

Loop

도구 결과로 다음 단계

내장 도구 카탈로그

도구별 역할과 입출력

Tool	역할	주요 입력
Read	파일 내용 읽기 (텍스트/이미지/PDF)	file_path, range
Edit	파일 부분 수정 (Find & Replace)	old_str, new_str
Write	신규 파일 생성	file_path, content
Bash	셸 명령 실행	command, timeout
Grep / Glob	검색 (정규식/파일 패턴)	pattern, path
WebSearch/Fetch	웹 검색/페이지 가져오기	query, url

Atomic

단일 책임 도구

JSON I/O

구조화 인터페이스

Sandboxed

격리된 실행 환경

Auditable

호출 내역 추적 가능

MCP Tool Integration

Model Context Protocol로 외부 도구 확장

MCP

Model Context Protocol은 Anthropic이 개발한 표준 프로토콜로, 외부 시스템이 LLM에 도구를 제공하는 방식을 통일.

Slack, Jira, GitHub 등 다양한 통합이 가능.

STDIO

표준 입출력 기반

MCP 서버를 로컬 서브프로세스로 띄워
stdio로 통신.
가장 일반적인 방식.

HTTP

HTTP/SSE 기반

원격 MCP 서버를 HTTP로 호출.
조직 차원에서 호스팅하는 도구에 적합.

WEBSOCKET

WS 기반 양방향

실시간성이 중요한 도구에 사용.
채팅, 알림 같은 인터랙티브 도구에 적합.

MCP

Anthropic 표준

stdio

기본 전송

HTTP/SSE

원격 호스팅용

Ecosystem

확장 가능 생태계

권한 모델 (Permission Model)

도구 호출 통제와 안전성

```
# settings.json
{
  "permissions": {
    "allow": [
      "Read(**)",
      "Bash(npm test:*)",
      "Bash(git status)",
      "Bash(git diff:*)"
    ],
    "ask": [
      "Edit(**)",
      "Write(**)"
    ],
    "deny": [
      "Bash(rm -rf:*)",
      "Bash(sudo:*)",
      "Bash(curl * | sh)"
    ]
  }
}
```

settings.json

allow

자동 허용

ask

사용자 확인

deny

거부

pattern

그룹 패턴 매칭

Context Window Management

200K 토큰을 효율적으로 활용하는 법

TOTAL

200K 토큰

기본 컨텍스트 윈도우.
약 50만 한국어 자, 영문 도서
200페이지 수준.

AUTO

/compact

오래된 메시지를 요약해 컨
텍스트를 압축.
자동 또는 수동 실행 가능.

RESET

/clear

컨텍스트를 완전히 비우고
새로 시작.
작업 전환 시 권장.

GUARD

CLAUDE.md

핵심 정보를 영구 메모리에
저장해 매 세션에서 자동 로
드.

BEST PRACTICE

큰 작업은 여러 세션으로 나누고 핵심 컨텍스트는 CLAUDE.md에 저장하세요.
막무가내 /compact보다 의도적 /clear가 더 효과적일 때가 많습니다.

200K

기본 윈도우

/compact

자동 요약

/clear

완전 초기화

CLAUDE.md

영구 메모리

Context Compaction 상세

/compact 명령의 내부 동작

STEP 01

컨텍스트
사용량 점검

STEP 02

요약 대상
메시지 선정

STEP 03

Claude로
요약 생성

STEP 04

원본 메시지
요약본으로 대체

STEP 05

컨텍스트
재구성 완료

WHEN TO COMPACT

컨텍스트의 70% 이상이 사용될 때 자동으로 안내되거나, 사용자가 /compact 명령으로 명시적으로 실행할 수 있음.

TRADE-OFF

요약된 내용은 원본 디테일을 일부 잃으므로, 정밀한 코드 디테일이 필요한 작업에서는 /clear와 CLAUDE.md 조합이 더 안전.

70%

자동 안내 임계값

LLM

요약 생성 모델

Lossy

디테일 손실 발생

Reversible

되돌릴 수 없음

Security Architecture

계층별 보안 통제

NETWORK

네트워크 계층

TLS 1.2+, 인증서 검증, 화이트리스트
도메인, 사내 프록시 지원

AUTH

인증/인가 계층

자격증명 안전 저장, IAM Role 자동 감
지, OAuth 토큰 회전

TOOL

도구 실행 계층

권한 규칙, 위험 명령 차단, 사용자 승인
강제, 감사 로그

DEFENSE IN DEPTH

단일 계층 우회만으로 시스템이 침해되지 않도록 다층 방어를 설계합니다. 엔터프라이즈는 추가로 Hooks와 정책을 결합합니다.

TLS

전송 암호화

IAM/OAuth

신원 검증

Permission

권한 통제

Audit

감사 로그

네트워크 트래픽 패턴

방화벽/프록시 설정에 필요한 정보

Endpoint	용도	Port
api.anthropic.com	Direct API 모드	443
console.anthropic.com	OAuth 로그인	443
bedrock-runtime.{region}.aws.com	AWS Bedrock 모드	443
{region}-aiplatform.googleapis.com	GCP Vertex 모드	443
github.com / npmjs.com	설치/업데이트 시	443

443	Outbound	Whitelist	Proxy
HTTPS only	Egress 방향만	엔드포인트 제한 가능	HTTP_PROXY 지원

Logging & Telemetry

로컬 로그와 사용량 메트릭

LOCAL

로컬 로그

~/.claude/logs 디렉토리에
세션 로그 저장.
디버깅용 stack trace 포함.

USAGE

사용량 메트릭

토큰 사용량, 도구 호출 횟수
, 응답 시간을 /cost 명령으
로 확인.

AUDIT

감사 로그 (Ent)

엔터프라이즈에서 OTel
exporter로 사내 SIEM에
감사 로그를 전송 가능.

TRACE

디버그 트레이스

--debug 또는
CLAUDE_DEBUG=1 로 상세
트레이스 활성화.

NOTE

기본 텔레메트리는 로컬에만 남으며, 사용자 코드 내용은 Anthropic에 전송되지 않으며, 엔터프라이즈 옵션 사용 시 정책에 따라 조정.

~/.claude/logs

로컬 로그 경로

/cost

사용량 확인 명령

OTel

엔터프라이즈 표준

--debug

상세 트레이스

Part 2 Recap

아키텍처 핵심 정리

TAKEAWAY 1

3계층 클라이언트

CLI, IDE, SDK 세 진입점이
동일한 Core를 공유.

TAKEAWAY 2

모델 자동 라우팅

작업 복잡도에 따라
Sonnet/Opus/Haiku가 자
동 선택.

TAKEAWAY 3

도구 + 권한 모델

내장 도구와 MCP,
allow/ask/deny 권한이 안
전한 자동화 기반.

TAKEAWAY 4

다층 보안

네트워크, 인증, 도구, 감사
4계층의 Defense in
Depth.

NEXT - PART 3

Part 3은 Installation입니다. macOS, Linux, Windows, WSL2, Docker, 컨테이너 환경별 22가지 설치 시나리오를 단계별로 다룹니다.

Part 2

완료

Part 3

Installation (22p)

Part 4

Authentication (18p)

~3h

Chapter 1 잔여

CHAPTER 01 - Part 03

Overview - Installation

Claude Code의 정체성, 아키텍처, 설치, 인증, 핵심 기능, 워크플로 패턴까지



Part 3 / Installation

환경별 설치 가이드와 트러블슈팅

PART OBJECTIVE

본인의 운영체제, 패키지 매니저, 네트워크 환경에 맞춰 Claude Code를 설치하고 검증하는 방법을 학습합니다. 트러블슈팅까지 포함.

OS

macOS/Linux/Windows

3대 운영체제 + WSL2 + Docker + Devcontainer 환경.

INSTALLER

npm vs Native

npm 글로벌 설치와 native installer 두 가지 방식을 비교/선택.

NODE MGR

nvm/fnm/Volta

Node.js 버전 매니저별 설치 가이드를 제공.

TROUBLE

트러블슈팅

권한 오류, 프록시, 인증서, 폰트 등 자주 발생하는 문제 해결법.

22 Slides

Part 3 분량

~45 min

예상 강의 시간

6+ Env

지원 환경

Lab 1

실습 포함

시스템 요구사항

하드웨어와 소프트웨어 요건

항목	최소 요구사항	권장
OS	macOS 12+, Ubuntu 20.04+, Win11+WSL2	최신 안정 버전
CPU	x86_64 또는 ARM64	4코어 이상
RAM	4GB	8GB 이상
Disk	500MB	5GB+ (캐시/로그)
Node.js	v18 LTS (npm 설치 시)	v20 LTS+
Network	HTTPS 443 outbound	안정된 대역폭

4GB
최소 RAM

8GB+
권장 RAM

Node 18+
최소 Node

TLS 1.2+
필수 통신 요건

설치 방법 비교

npm vs Native vs Docker

NPM

npm 글로벌

Node.js 환경 보유 시 가장 쉬움. 업그
레이드 간편. 사내 npm 미러 활용 가능.

NATIVE

Native Installer

Node 없이 사용 가능.
Homebrew/curl/MSI로 배포. 엔터프
라이즈 표준 배포에 적합.

DOCKER

Docker / Devcontainer

격리 환경 보장. CI/CD에 적합. 사전 구
성된 공식 이미지 제공.

CHOICE GUIDE

Node 보유 개인은 npm, Node 없는 표준 배포는 Native, 격리/자동화는 Docker. 모든 옵션 모두 동일한 Claude Code를 설치.

npm

개발자 친화

native

표준 배포

docker

자동화/격리

devcontainer

VS Code 통합

npm 설치 / macOS & Linux

단계별 가이드

macOS / Linux

```
# 1. Node.js 18 LTS 이상 설치 확인
node --version # v18.x.x 이상이어야 함

# 2. Claude Code 글로벌 설치
npm install -g @anthropic-ai/claude-code

# 3. 설치 검증
claude --version
claude doctor # 환경 자가 진단

# 4. 첫 실행
cd /path/to/your/project
claude
# > 처음에 인증 안내가 표시됩니다.

# (선택) 권한 오류 해결
npm config get prefix
# /usr/local 이면 sudo 필요할 수 있음
# nvm/fnm 사용 권장 (sudo 불필요)
```

-g
글로벌 설치 플래그

doctor
자가 진단 명령

nvm
sudo 회피 권장

prefix
설치 경로 확인

npm 설치 / Ubuntu & Debian

apt와 NodeSource 활용

Ubuntu / Debian

```
# 1. apt 기본 저장소의 Node는 오래된 경우 많음 - NodeSource 권장
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
```

```
# 2. 설치 확인
node --version # v20.x.x
npm --version # v10.x.x
```

```
# 3. Claude Code 설치 (sudo 불필요한 prefix 설정)
mkdir -p ~/.npm-global
npm config set prefix '~/.npm-global'
echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

```
# 4. Claude Code 글로벌 설치
npm install -g @anthropic-ai/claude-code
claude --version
```

NodeSource

최신 Node 저장소

prefix

사용자 디렉토리 설정

PATH

환경변수 갱신

No sudo

sudo 없이 설치

npm 설치 / RHEL & Fedora

dnf와 NodeSource

RHEL / Fedora

```
# 1. NodeSource 저장소 추가 (RHEL/CentOS/Rocky/Fedora)
curl -fsSL https://rpm.nodesource.com/setup_20.x | sudo bash -
sudo dnf install -y nodejs

# 2. 또는 dnf 모듈을 통한 설치 (RHEL 9+)
sudo dnf module install nodejs:20/common

# 3. 설치 확인
node --version
npm --version

# 4. 사용자 prefix 설정 후 Claude Code 설치
mkdir -p ~/.npm-global
npm config set prefix '~/.npm-global'
echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.bashrc
source ~/.bashrc

npm install -g @anthropic-ai/claude-code
```

dnf

RHEL 9+ 패키지 매니저

module

dnf 모듈 스트림

NodeSource

rpm 저장소

SELinux

정책 확인 권장

npm 설치 / Windows

PowerShell & Native Windows

Windows PowerShell

```
# 1. Node.js 설치 - 권장: nvm-windows 사용
# https://github.com/coreybutler/nvm-windows/releases
nvm install 20
nvm use 20

# 2. PowerShell에서 설치
npm install -g @anthropic-ai/claude-code

# 3. 실행 정책 확인 (필요 시)
Get-ExecutionPolicy
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser

# 4. PATH 확인 및 검증
$env:Path -split ';' | Select-String 'npm'
claude --version

# (대안) WSL2를 사용한다면 Linux와 동일한 절차
# wsl.exe 실행 후 위 Linux 가이드 참조
```

nvm-windows

추천 Node 매니저

PowerShell

권장 셸

WSL2

대안 권장

RemoteSigned

실행 정책

Native Installer / macOS

Homebrew를 통한 표준 설치

macOS Native

```
# 1. Homebrew 확인 (없다면 설치)
brew --version
# Homebrew가 없는 경우:
# /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 2. Anthropic tap 추가 후 설치
brew install --cask claude-code

# 3. 또는 직접 다운로드 설치
curl -fsSL https://claude.ai/install.sh | bash

# 4. 검증
claude --version
which claude      # /opt/homebrew/bin/claude (Apple Silicon)
                  # /usr/local/bin/claude   (Intel)

# 5. 업그레이드
brew upgrade --cask claude-code
# 또는: claude update
```

Homebrew

표준 패키지 매니저

cask

바이너리 형태 설치

No Node

Node.js 불필요

upgrade

brew upgrade

Native Installer / Linux

curl 스크립트와 패키지 매니저

Linux Native

```
# 1. 공식 install.sh 스크립트 (curl)
curl -fsSL https://claude.ai/install.sh | bash

# 스크립트 동작:
#   - 사용자 OS/아키텍처 감지 (linux-x64, linux-arm64)
#   - ~/.claude/bin 에 claude 바이너리 다운로드
#   - PATH 자동 추가 (.bashrc / .zshrc)

# 2. 검증
source ~/.bashrc
claude --version

# 3. APT/RPM 기반 패키지 (엔터프라이즈 미러)
# Anthropic이 제공하는 deb/rpm 패키지를 사내 미러에 배포
# 자세한 절차는 Chapter 3 Admin Setup에서 다룸

# 4. 업데이트
claude update
```

install.sh

공식 스크립트

~/.claude/bin

설치 경로

deb/rpm

엔터프라이즈 패키지

claude update

업그레이드 명령

Native Installer / Windows

MSI 설치와 WSL2 권장

Windows Native

```
# 1. 공식 MSI 설치 - 권장 (관리자 권한 필요)
# https://claude.ai/download/windows.msi 에서 다운로드 후 실행
# 또는 PowerShell:
Invoke-WebRequest -Uri https://claude.ai/install.ps1 -OutFile install.ps1
.\install.ps1

# 2. WinGet (Windows Package Manager) - Windows 10/11
winget install Anthropic.ClaudeCode

# 3. Scoop (서드파티 패키지 매니저)
scoop bucket add claude https://github.com/anthropics/scoop-claude
scoop install claude-code

# 4. 검증
claude --version

# 5. (강력 권장) WSL2 사용 - Linux 가이드 참조
wsl --install -d Ubuntu-22.04
```

MSI

표준 설치 패키지

WinGet

Win10/11 공식

Scoop

서드파티 옵션

WSL2

권장 환경

Node Manager / nvm

macOS & Linux에서의 권장 Node 매니저

```
# 1. nvm 설치
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc # 또는 ~/.zshrc
```

```
# 2. Node 18, 20, 22 설치
nvm install 18
nvm install 20
nvm install 22
```

```
# 3. 기본 버전 지정
nvm alias default 20
nvm use default
```

```
# 4. 프로젝트별 .nvmrc
echo "20" > .nvmrc
nvm use # .nvmrc 자동 인식
```

```
# 5. Claude Code 글로벌 설치 (sudo 불필요)
npm install -g @anthropic-ai/claude-code
```

nvm

nvm

최다 사용 매니저

.nvmrc

프로젝트별 고정

shell

셸 함수 기반

No sudo

sudo 불필요

Node Manager / fnm

빠르고 가벼운 nvm 대안

```
# 1. fnm 설치 (Rust 기반, nvm보다 훨씬 빠름)
curl -fsSL https://fnm.vercel.app/install | bash
# 또는 Homebrew: brew install fnm

# 2. 셸 통합 활성화
echo 'eval "$(fnm env --use-on-cd)"' >> ~/.bashrc
source ~/.bashrc

# 3. Node 설치
fnm install 20
fnm install --lts

# 4. 기본 버전
fnm default 20

# 5. .nvmrc / .node-version 자동 인식 (cd 시)
echo "20" > .node-version
cd . # 자동 전환 동작 확인

# 6. Claude Code 설치
npm install -g @anthropic-ai/claude-code
```

fnm

fnm

Fast Node Manager

Rust

구현 언어

10-100x

nvm 대비 속도

use-on-cd

cd 시 자동 전환

Node Manager / Volta

JS 도구 전체를 관리하는 매니저

Volta

```
# 1. Volta 설치
curl https://get.volta.sh | bash
# 또는 Windows: winget install Volta.Volta

# 2. Node 설치 + 기본 설정
volta install node@20
volta install npm@10

# 3. 프로젝트별 도구 고정 (package.json에 자동 기록)
cd my-project
volta pin node@20
volta pin npm@10

# 4. 패키지 도구도 함께 관리
volta install @anthropic-ai/claude-code
# /home/user/.volta/bin/claude

# 5. 검증
volta list
claude --version
```

Volta
JS 도구 매니저

pin
프로젝트 버전 고정

package.json
버전 자동 기록

Cross-OS
mac/Linux/Windows

Docker 컨테이너

Anthropic 공식 이미지로 격리 실행

Docker

```
# 1. 공식 이미지 풀
docker pull anthropic/claude-code:latest

# 2. 단일 명령 실행 (헤드리스)
docker run --rm -it \
  -v $(pwd):/workspace \
  -e ANTHROPIC_API_KEY="$ANTHROPIC_API_KEY" \
  anthropic/claude-code:latest \
  -p "package.json에서 outdated 패키지를 알려 주세요"

# 3. 대화형 세션
docker run --rm -it \
  -v $(pwd):/workspace \
  -v $HOME/.claude:/root/.claude \
  -e ANTHROPIC_API_KEY \
  anthropic/claude-code:latest

# 4. docker-compose.yml 활용
docker compose up claude-code
```

anthropic/
공식 이미지 prefix

-v
워크스페이스 마운트

-e KEY
환경변수 주입

compose
docker compose 호환

Docker 사용자 정의 이미지

사내 도구 추가한 베이스 이미지 구축

Dockerfile

```
# Dockerfile
FROM anthropic/claude-code:latest

# 사내 자주 쓰는 CLI 추가
RUN apt-get update && apt-get install -y \
    jq ripgrep awscli git-lfs \
    && rm -rf /var/lib/apt/lists/*

# 사내 CLAUDE.md 템플릿 사전 배치
COPY company-CLAUDE.md /etc/claude/CLAUDE.md.template

# 사내 MCP 서버 설정
COPY mcp-config.json /etc/claude/mcp.json

ENV CLAUDE_CONFIG_DIR=/etc/claude
ENV NODE_TLS_REJECT_UNAUTHORIZED=1

WORKDIR /workspace

ENTRYPOINT ["claude"]
```

FROM

공식 베이스 활용

CLAUDE.md

사내 템플릿 사전 배치

MCP

MCP 사전 설정

ENV

환경변수 사전 설정

WSL2 환경 설정

Windows에서 Linux 환경 활용

WSL2

```
# 1. WSL2 설치 (PowerShell 관리자 권한)
wsl --install -d Ubuntu-22.04
# 재부팅 후 사용자 생성

# 2. WSL 내부에서 Ubuntu 업데이트
sudo apt update && sudo apt upgrade -y

# 3. Node.js 20 설치 (NodeSource)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# 4. npm prefix 설정 후 Claude Code 설치
mkdir -p ~/.npm-global
npm config set prefix '~/.npm-global'
echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
npm install -g @anthropic-ai/claude-code

# 5. Windows-WSL 파일 시스템 주의
# /mnt/c/Users/... 보다 ~/projects 권장 (성능)
```

WSL2

Windows Subsystem

Ubuntu

권장 배포판

/mnt/c

Windows 파일 접근

~/projects

권장 작업 경로

Devcontainer 설정

VS Code 통합 표준 개발 환경

```
// .devcontainer/devcontainer.json
{
  "name": "Claude Code Dev",
  "image": "anthropic/claude-code:latest",
  "features": {
    "ghcr.io/devcontainers/features/git:1": {},
    "ghcr.io/devcontainers/features/github-cli:1": {}
  },
  "containerEnv": {
    "ANTHROPIC_API_KEY": "${localEnv:ANTHROPIC_API_KEY}"
  },
  "mounts": [
    "source=${localEnv:HOME}/.claude,target=/root/.claude,type=bind"
  ],
  "postCreateCommand": "claude doctor",
  "customizations": {
    "vscode": {
      "extensions": ["anthropic.claude-code"]
    }
  }
}
```

Devcontainer

VS Code 표준

features

Git/gh 자동 설치

mounts

로컬 ~/.claude 공유

postCreate

doctor 자동 실행

설치 검증 / claude --version

기본 검증 단계

verification

```
# 1. 버전 확인
$ claude --version
claude-code 1.x.x (build 20251101)

# 2. 도움말 확인
$ claude --help
Usage: claude [options] [prompt]

Options:
  -p, --prompt <text>      One-shot prompt (headless)
  -c, --continue            Continue last session
  -r, --resume <id>        Resume specific session
  --model <name>            Override model (e.g., sonnet, opus)
  --output-format <fmt>    text | json | stream-json
  --allowed-tools <list>   Comma-separated tool names
  --dangerously-skip-permissions (위험: 모든 권한 부여)
  --debug                  Verbose tracing
```

--version

버전 확인

--help

도움말 출력

-p

헤드리스 프롬프트

--debug

상세 트레이스

설치 검증 / claude doctor

자가 진단으로 환경 확인

```
$ claude doctor
```

claude doctor

Claude Code Doctor

System

OS	macOS 14.6 (arm64)	OK
Node	v20.10.0	OK
Disk free	12.3 GB	OK
RAM	16 GB	OK

Network

api.anthropic.com	reachable (43 ms)	OK
Outbound HTTPS	allowed	OK
Proxy	none detected	OK

Auth

Credentials	ANTHROPIC_API_KEY (env)	OK
Token valid	yes (Pro plan)	OK

Config

Settings file	~/.claude/settings.json	OK
CLAUDE.md	not found (project root)	i
MCP servers	2 configured	OK

Issues: 0 errors, 1 info

doctor

--verbose

OK

i

자가 진단

상세 출력

정상

안내 (경고 아님)

업그레이드 / claude update

안전한 버전 관리

```
# 1. 최신 버전 확인
claude --version
claude update --check
# Latest: 1.5.0 / Installed: 1.4.2 -> update available

# 2. 자동 업그레이드
claude update
# 패키지 매니저에 따라 자동 적용 (npm/brew/native)

# 3. npm으로 설치한 경우 직접 명령도 가능
npm update -g @anthropic-ai/claude-code

# 4. brew로 설치한 경우
brew upgrade --cask claude-code

# 5. 특정 버전 고정 (조직 표준)
npm install -g @anthropic-ai/claude-code@1.4.2

# 6. 자동 업데이트 체크 비활성화 (CI 환경)
export CLAUDE_DISABLE_UPDATE_CHECK=1
```

upgrade

update

자동 업그레이드

--check

버전 확인만

pin

버전 고정 권장

CLAUDE_DISABLE

체크 비활성화

제거 / Uninstall

완전한 클린업 절차

uninstall

```
# 1. 설치 방식별 제거

# npm으로 설치한 경우
npm uninstall -g @anthropic-ai/claude-code

# brew로 설치한 경우
brew uninstall --cask claude-code

# native install.sh로 설치한 경우
rm -rf ~/.claude/bin/claude
# PATH에서 ~/.claude/bin 제거 (~/.bashrc 편집)

# Windows MSI로 설치한 경우
# 제어판 > 프로그램 추가/제거 > Claude Code 제거

# 2. 사용자 설정/세션 클린업 (선택)
rm -rf ~/.claude/sessions # 세션 기록 삭제
rm -rf ~/.claude/logs     # 로그 삭제
rm -f ~/.claude/settings.json # 설정 삭제 (주의)

# 3. 모든 흔적 완전 제거
rm -rf ~/.claude # 모든 데이터 삭제
```

uninstall

패키지 제거

~/.claude

사용자 데이터

sessions

세션 기록

settings

설정 파일

트러블슈팅

자주 발생하는 문제와 해결법

증상	원인	해결
EACCES on npm install -g	글로벌 npm 권한 부족	prefix 변경 또는 nvm
SSL/TLS 인증서 오류	사내 프록시의 자체 인증서	NODE_EXTRA_CA_CERTS
api.anthropic.com unreachable	방화벽/프록시 차단	HTTPS_PROXY 설정
한국어 깨짐	로케일/UTF-8 미설정	export LANG=ko_KR.UTF-8
Bash 도구 호출 실패	권한 규칙 차단	settings.json 조정
claude: command not found	PATH 미반영	source 또는 새 셸 재시작

doctor

먼저 실행

--debug

트레이스 활성화

logs

~/claude/logs 확인

support

미해결 시 공식 채널

CHAPTER 01 - Part 04

Overview – Authentication

Claude Code의 정체성, 아키텍처, 설치, 인증, 핵심 기능, 워크플로 패턴까지



Part 4 / Authentication

4가지 인증 옵션의 선택과 구성

PART OBJECTIVE

본인 또는 조직 환경에 최적인 인증 방법을 선택하고, 자격증명을 안전하게 구성하며, 운영 중 자격증명을 회전/관리하는 방법을 학습합니다.

METHOD 1

Claude Pro/Max
OAuth

개인 구독 계정을 브라우저
OAuth로 연결. 가장 빠른
시작.

METHOD 2

Anthropic API Key

Console에서 발급한 API
Key 사용. 종량제 환경에 적
합.

METHOD 3

AWS Bedrock

IAM 자격증명으로 AWS 청
구 경유. 엔터프라이즈 표준.

METHOD 4

Google Vertex AI

ADC 자격증명으로 GCP 청
구 경유. GCP 표준 통합.

18 Slides

Part 4 분량

~35 min

예상 강의 시간

4 Methods

인증 옵션

Lab 2

실습 포함

Method 1 / Claude Pro/Max OAuth

구독 계정 브라우저 OAuth 인증

OAuth Flow

```
# 1. 첫 실행 (또는 /login 명령)
$ claude
> Welcome to Claude Code! Please sign in to continue.
> Press Enter to open the browser for authentication...
[Enter]

# 2. 브라우저에서 OAuth 동의 (자동으로 콘솔 사이트 열림)
#   - Claude.ai 계정 로그인
#   - "Authorize Claude Code" 클릭
#   - 임시 코드가 CLI로 자동 전달됨

# 3. 인증 완료 후 자동으로 CLI에 복귀
> Successfully signed in as user@example.com
> Plan: Claude Max

# 4. 로그아웃
/logout
```

/login
로그인 명령

/logout
로그아웃

Browser
OAuth UI

PKCE
안전한 OAuth 표준

Pro/Max 사용량 한도

구독 플랜별 토큰/요청 한도

Plan	한도 특성	대상
Claude Pro	5시간 단위 메시지 한도, 일일 사용량 캡	개인 일상 사용
Claude Max	Pro 대비 5배~20배 한도, 우선 액세스	파워 유저
Team	시트 단위, 조직 청구 통합	소규모 팀
Enterprise	맞춤 한도, SLA, SSO, BAA	대규모 조직

IMPORTANT

구독 플랜은 일정 한도 도달 시 일시적으로 응답이 제한될 수 있으며, 자동화 워크로드는 API Key 또는 Bedrock 경로가 더 적합.

5h window

Pro 한도 단위

5x-20x

Max 증분

SSO

Team/Ent 표준

SLA

Enterprise 옵션

Pro/Max 사용량 모니터링

/cost 명령과 일일 캡 추적

```
# 1. 현재 세션 사용량 확인
$ /cost
Session usage
  Input tokens: 45,231
  Output tokens: 12,845
  Tool calls: 87
  Subagent calls: 3
  Cache reads: 8,910

Daily plan usage (Claude Max)
  Used: 2,341 / 10,000 messages (23%)
  Resets: in 3h 27m

# 2. 누적 사용 내역 (Anthropic 콘솔)
# https://console.anthropic.com/settings/usage

# 3. 한도 경고
> [WARN] You've used 85% of your Pro window.
> Consider /clear or switching to API key.
```

usage

/cost

세션별 사용량

Console

누적 대시보드

85%

경고 임계값

Reset

한도 리셋 시간 표시

Method 2 / Anthropic API Key

Console에서 발급한 키로 종량제 사용

API Key

```
# 1. 환경변수 방식 (가장 일반적)
export ANTHROPIC_API_KEY="sk-ant-api03-..."
claude

# 2. 영구 저장 (~/.bashrc 또는 ~/.zshrc)
echo 'export ANTHROPIC_API_KEY="sk-ant-api03-..."' >> ~/.zshrc
source ~/.zshrc

# 3. macOS Keychain 사용 (권장)
security add-generic-password \
  -s anthropic_api_key -a "$USER" -w "sk-ant-..."
# 사용 시:
export ANTHROPIC_API_KEY=$(security find-generic-password \
  -s anthropic_api_key -a "$USER" -w)

# 4. 1Password CLI 통합 (팀 권장)
export ANTHROPIC_API_KEY=$(op read "op://Personal/anthropic/key")
```

sk-ant-
Key 접두사

env
환경변수 방식

Keychain
안전 저장

1Password
팀 권장 통합

Console API Key 발급

단계별 발급 절차

STEP 01

console.anthropic.com
로그인

STEP 02

Settings
API Keys

STEP 03

Create Key
이름 + 권한 지정

STEP 04

Key 값 즉시 복사
(한 번만 표시)

STEP 05

안전한 저장소에
보관

BEST PRACTICE

Key 이름에는 용도와 환경을 명시 (예: "claude-code/dev/user@org.com").

Key 발급 후에는 다시 표시되지 않으므로 즉시 안전한 저장소로 옮겨야 합니다 !!!

KEY ROTATION

API Key는 90일 주기로 회전하는 것이 권장.

Console에서 새 Key 생성 후 이전 Key를 폐기하는 식으로 안전 회전이 가능.

Console

발급 위치

Name+Tag

식별성 부여

Show Once

한 번만 노출

90 days

권장 회전 주기

API Key 환경변수 설정

다양한 셸과 환경 통합

env vars

```
# Bash / Zsh
export ANTHROPIC_API_KEY="sk-ant-..."

# Fish shell
set -gx ANTHROPIC_API_KEY "sk-ant-..."

# PowerShell
$env:ANTHROPIC_API_KEY = "sk-ant-..."

# Windows cmd (영구 저장)
setx ANTHROPIC_API_KEY "sk-ant-..."

# direnv (.envrc - 프로젝트별)
export ANTHROPIC_API_KEY="sk-ant-..."
# .envrc는 .gitignore에 반드시 추가!

# .env 파일 (dotenv 라이브러리와 사용 시)
ANTHROPIC_API_KEY=sk-ant-...

# Docker / Kubernetes
docker run -e ANTHROPIC_API_KEY ...
kubectl create secret generic claude-key \
  --from-literal=ANTHROPIC_API_KEY="sk-ant-..."
```

export
POSIX 셸

setx
Windows cmd

direnv
프로젝트별

k8s secret
Kubernetes

API Key 보안 베스트 프랙티스

7가지 핵심 수칙

RULE 1

키를 코드에 하드코딩
금지

소스 코드 저장소에 키가 한 번
이라도 들어가면 즉시 폐기.

RULE 2

환경별 키 분리

dev, staging, prod 각각
다른 키를 발급해 폭발 반경
을 줄임.

RULE 3

시크릿 매니저 사용

1Password, Vault, AWS
Secrets Manager 등을 활
용.

RULE 4

회전 주기 준수

90일 주기로 회전. 직원 퇴
사/이동 시 즉시 회전.

ADDITIONAL RULES

Rule 5: .gitignore에 .env/.envrc 포함 , Rule 6: GitHub secret 스캔 활성화 , Rule 7: 사용량 알림 설정 (이상 사용 감지)

No hardcode

코드 내 평문 금지

Per-env

환경별 분리

Vault

시크릿 매니저

90d

회전 주기

Method 3 / AWS Bedrock

AWS 청구 경로로 Claude 사용

AWS Bedrock

```
# 1. Bedrock 모드 활성화
export CLAUDE_CODE_USE_BEDROCK=1
export AWS_REGION=us-west-2

# 2. AWS 자격증명 (택1)
# (a) AWS_PROFILE 사용
export AWS_PROFILE=anthropic-prod

# (b) AWS_ACCESS_KEY_ID + AWS_SECRET_ACCESS_KEY
export AWS_ACCESS_KEY_ID=AKIA...
export AWS_SECRET_ACCESS_KEY=...

# (c) EC2/EKS IAM Role - 자동 감지

# 3. 모델 ID (Bedrock 형식)
export ANTHROPIC_MODEL=anthropic.claude-sonnet-4-5-20250929-v1:0

# 4. 실행
claude
# > Successfully authenticated via AWS Bedrock (us-west-2)
```

USE_BEDROCK

모드 활성화

AWS_REGION

리전 지정

IAM Role

자동 감지

Model ID

Bedrock 모델 ID

Bedrock 모델 액세스 신청

AWS Console에서 모델 활성화

STEP 01

AWS Console
Bedrock 이동

STEP 02

Model access
메뉴 선택

STEP 03

Anthropic 모델
Request access

STEP 04

약관 동의 +
사용 사례 기술

STEP 05

자동/수동
승인

AVAILABLE MODELS

Claude Sonnet, Opus, Haiku 패밀리가 모두 제공. 신규 모델은 출시 후 일정 기간 지나 Bedrock에 등록되며, 일부 리전에서만 제공될 수 있으니 리전 선정에 주의가 필요.

TIPS

계정당 한 번 승인되면 동일 계정 내 모든 IAM 사용자가 해당 모델을 사용할 수 있음.

신규 리전을 추가하려면 그 리전에서 별도 신청이 필요할 수 있음.

Console

신청 위치

us-west-2

주요 리전

Region

리전별 신청

Free

신청 자체는 무료

Bedrock 환경변수 상세

모든 옵션과 모델 ID 매핑

환경변수	용도	예시
CLAUDE_CODE_USE_BEDROCK	Bedrock 모드 활성화	1
AWS_REGION	Bedrock 리전	us-west-2
AWS_PROFILE	credentials 프로필	anthropic-prod
ANTHROPIC_MODEL	기본 모델 ID	anthropic.claude-sonnet-4-5-...
ANTHROPIC_SMALL_FAST_MODEL	Haiku 모델 ID	anthropic.claude-haiku-...
AWS_BEDROCK_INFERENCE_PROFILE	추론 프로필 (CRIS)	us.anthropic.claude-...

Region

리전 정확히 지정

CRIS

Cross-Region Inference

Profile

권장 자격증명

Two models

메인+Haiku 별도

Bedrock IAM 정책

Claude Code에 필요한 최소 권한

IAM Policy (least privilege)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ClaudeCodeBedrockInvoke",
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream"
      ],
      "Resource": [
        "arn:aws:bedrock:us-west-2::foundation-model/anthropic.claude-sonnet-*",
        "arn:aws:bedrock:us-west-2::foundation-model/anthropic.claude-haiku-*",
        "arn:aws:bedrock:us-east-1::inference-profile/us.anthropic.claude-*"
      ]
    },
    {
      "Sid": "ListModelsOptional",
      "Effect": "Allow",
      "Action": ["bedrock:ListFoundationModels"],
      "Resource": "*"
    }
  ]
}
```

InvokeModel

필수 액션

Stream

스트리밍 액션

ARN

리소스 정확 지정

CRIS

inference-profile ARN

Bedrock 리전 전략

리전 선정과 CRIS 활용

PRIMARY

us-west-2

Claude 모델 가장 빠른 출시.
대부분의 최신 모델이 우선 제공.

SECONDARY

us-east-1

두 번째로 빠른 출시 리전.
미국 동부 사용자에게 지연 시간 우위.

CRIS

us.anthropic.*

Cross-Region Inference로 us-east-1
+ us-west-2를 묶어 처리량 증대.

KOREA-SPECIFIC

한국 사용자는 ap-northeast-1 (도쿄) 또는 ap-northeast-2 (서울) 사용 시 일부 모델만 제공될 수 있습니다.
최신 모델은 us-west-2를 권장하며, 지연 시간은 CloudFront/Bedrock GA 가속화로 보완합니다.

us-west-2

권장 리전

CRIS

처리량 + 가용성

ap-northeast-2

한국 리전 제약

Latency

지연 시간 고려

자격증명 우선순위

여러 방법이 충돌할 때의 순서

STEP 01

CLI 플래그
--api-key 등

STEP 02

환경변수
ANTHROPIC_API_KEY

STEP 03

~/.claude/keys
파일

STEP 04

시스템 자격증명
IAM/ADC

STEP 05

기본 OAuth
(저장된 토큰)

PRECEDENCE

왼쪽일수록 우선순위가 높음.

CLI 플래그가 가장 강력하며, 환경변수가 그 다음, 파일과 시스템 자격증명, 그리고 저장된 OAuth 토큰 순으로 적용.

DEBUG

어느 자격증명이 사용되는지 확인하려면 `claude doctor` 또는 `--debug` 플래그를 사용.

`settings.json`의 `authMode` 옵션으로 강제 지정도 가능.

CLI

최우선

Env

두 번째

File

세 번째

OAuth

폴백

/login, /logout 명령

세션 인증 전환과 종료

auth commands

```
# /login - 인증 시작 또는 재인증
> /login
> Choose authentication method:
>   1. OAuth (Pro/Max subscription)
>   2. Anthropic API Key (paste)
>   3. Use AWS Bedrock (existing credentials)
>   4. Use Vertex AI (existing credentials)
> Selected: 1

# /logout - 현재 세션 인증 해제
> /logout
> Successfully signed out. Tokens cleared.

# 인증 상태 확인
> /status
> Auth: OAuth (user@example.com)
> Plan: Claude Max
> Tokens used today: 23%

# 또는 명령행에서
$ claude --auth-info
```

/login

재인증

/logout

인증 해제

/status

현재 인증 상태

--auth-info

CLI 확인

CHAPTER 01- Part 05

Overview – Quick Start

Claude Code의 정체성, 아키텍처, 설치, 인증, 핵심 기능, 워크플로 패턴까지



Part 5 / Quick Start

첫 세션부터 시나리오 실습까지

PART OBJECTIVE

설치와 인증을 마친 후 본인의 프로젝트에 Claude Code를 적용하는 실전 흐름을 학습.

슬래시 명령, 시나리오 3종, 모드 전환을 다룸.

TOPIC 1

첫 세션과 슬래시 명령

claude 실행, /init, /help,
/status, /clear 등 필수 명
령 학습

TOPIC 2

프롬프트 작성법

명확한 지시, 컨텍스트 제공,
결과 검증을 위한 프롬프트
패턴

TOPIC 3

시나리오 3종

디버깅, 기능 추가, 리팩토링
시나리오를 단계별로 실습

TOPIC 4

Plan vs Auto 모드

두 모드의 차이와 전환 방법,
세션 관리 명령 학습

22 Slides

Part 5 분량

~40 min

예상 강의 시간

3 Scenarios

실전 시나리오

Lab 3-4

실습 포함

claude 명령어 옵션 개요

가장 자주 사용하는 플래그 10선

플래그	설명	용도
claude	대화형 REPL 시작	기본 사용
-p, --prompt <text>	헤드리스 일회성 실행	스크립트/CI
-c, --continue	직전 세션 이어가기	작업 재개
-r, --resume <id>	특정 세션 재개	세션 전환
--model <name>	모델 강제 지정	Opus/Haiku 선택
--allowed-tools <list>	허용 도구 화이트리스트	안전 제한
--output-format json	JSON 출력	파이프 처리
--debug	상세 트레이스	문제 진단

-p
헤드리스 핵심

-c
이어가기 핵심

--model
모델 전환

--debug
디버그

대화형 REPL 입문

첫 세션 동작 원리

REPL session

```
$ cd ~/projects/my-app  
$ claude  
Welcome to Claude Code 1.x.x
```

```
/help    Show available commands  
/init    Initialize CLAUDE.md  
/status  Show session status
```

> 안녕 Claude, 이 프로젝트 구조 좀 봐 줄래?

```
[Claude calls tools]  
• Read package.json  
• Glob src/**/*.{ts,tsx}  
• Read README.md
```

이 프로젝트는 Next.js 14 기반의 풀스택 앱이네요.
주요 구조는...

> 좋아. src/components 안에 새로운 Modal 컴포넌트를 만들어 줘.
Tailwind 사용하고, ESC로 닫히게.

claude

시작 명령

>

입력 프롬프트

/help

명령 목록

cwd

현재 디렉토리가 워크스페이스

첫 프롬프트 작성 팁

명확하고 효과적인 지시 방법

TIP 1

맥락 먼저

"무엇을 하고 싶은지" 보다 "현재 상황"을 먼저 알려 주면 Claude의 정확도가 높아짐.

TIP 2

구체적 요청

"개선해 줘" 보다 "타입 검사 통과 + 50% 이상 성능 향상" 같이 측정 가능한 목표 제시.

TIP 3

예시 제공

원하는 결과의 예시 입출력이나 유사 코드를 함께 제공하면 더 정확해짐.

TIP 4

제약 명시

"이 파일은 건드리지 마" 같이 금지 사항을 명시하면 의도와 다른 변경을 방지.

GENERAL RULE

맥락 → 목표 → 제약 → 검증 방법의 4단계 구조로 프롬프트를 작성하면 만족도가 상승.

Context

맥락 먼저

Specific

구체적 목표

Examples

예시 제공

Constraints

제약 명시

명확한 vs 모호한 지시

실제 예시 비교

BAD

모호한 지시

"코드 좀 봐 줘"

"버그 고쳐"

"성능 개선해"

"테스트 좀 짜 줘"

"리팩토링해"

→ 의도와 다른 작업이 발생

→ 여러 번 수정 반복

GOOD

명확한 지시

"src/api/users.ts 의 GET /users 핸들러를

검토하고, N+1 쿼리가 있다면

includes로 해결해 줘. 응답 시간 100ms 이내가 목표야."

→ 한 번에 의도된 결과 달성

→ 검증 가능한 목표

File path

대상 명시

Function

함수/엔드포인트 명시

Goal

측정 가능한 목표

Constraint

품질/스타일 제약

슬래시 명령 개요

REPL에서 사용 가능한 핵심 명령

명령	동작	사용 빈도
/help	사용 가능한 모든 명령 표시	필요 시
/init	CLAUDE.md 자동 생성	프로젝트 시작 시
/status	인증, 모델, 사용량 정보	점검 시
/clear	컨텍스트 완전 초기화	작업 전환 시
/compact	오래된 메시지 요약	컨텍스트 부족 시
/model <name>	모델 런타임 전환	복잡도 변경
/cost	토큰 사용량 확인	주기적 확인
/login, /logout	인증 전환/종료	환경 전환 시

/help

시작 첫 명령

/init

필수 1회

/clear

자주 사용

/cost

비용 추적

/init 명령 / 프로젝트 초기화

CLAUDE.md 자동 생성

```
$ claude
> /init

Analyzing project structure...
• Detected: Node.js + TypeScript + Next.js 14
• Detected: Tailwind CSS, Prisma ORM, PostgreSQL
• Detected: Jest + Playwright tests
• Detected: ESLint + Prettier

Generated CLAUDE.md (preview):
# My App - Claude Code Memory
## Tech Stack
- Next.js 14 (App Router)
- TypeScript, Tailwind, Prisma
## Conventions
- Components: src/components/**/*.tsx
- API routes: src/app/api/**/*.route.ts
- Tests: *.test.ts colocated
## Commands
- Dev: pnpm dev
- Test: pnpm test
- Build: pnpm build

Write to ./CLAUDE.md? (y/N) y
```

/init

/init

프로젝트 초기화

Auto-detect

자동 스택 감지

Preview

쓰기 전 미리 보기

Edit

이후 자유 편집

/help / 도움말

사용 가능한 명령 일람

> /help

/help

Available commands:

Session

/clear	Clear context and start fresh
/compact	Summarize older messages
/status	Show session info and usage

Configuration

/init	Initialize CLAUDE.md
/memory	Open CLAUDE.md in editor
/model <name>	Switch model (sonnet, opus, haiku)
/cost	Show token usage and cost

Auth

/login	Sign in
/logout	Sign out

Tools

/tools	List available tools
/mcp	List MCP servers

Other

/exit, /quit	End session
/help	Show this help

4 groups

명령 카테고리

Session

컨텍스트 제어

Config

동작 설정

Tools/MCP

도구 점검

/status / 세션 상태 확인

한눈에 보는 현재 상태

```
> /status
```

Session

```
ID:          sess_abc123
Started:     2026-05-24 09:32 (5m ago)
Working dir: ~/projects/my-app
CLAUDE.md:   loaded (450 lines)
```

Auth

```
Method:      OAuth (Claude Max)
User:        user@example.com
```

Model

```
Default:     claude-sonnet-4-5
Small/fast:  claude-haiku-4-5 (subagents)
```

Permissions

```
allow rules: 5
ask rules:   2
deny rules:  3
```

Usage (this session)

```
Input tokens: 12,403
Output tokens: 3,210
Context used:  8% of 200K
```

/status

ID

세션 식별자

CLAUDE.md

로드 상태

Model

활성 모델

Context %

윈도우 사용률

/clear / 컨텍스트 초기화

작업 전환 시 필수 명령

WHEN TO CLEAR

한 작업이 끝나고 전혀 다른 주제로 이동할 때 /clear를 사용해 컨텍스트를 완전히 비움. 이전 작업의 잔재가 새 작업에 영향을 주지 않게 함.

```
# 디버깅 작업 완료 후 새로운 기능 개발 시작
> 디버깅이 완료되었습니다. 테스트가 모두 통과합니다.
> /clear
> Context cleared. CLAUDE.md reloaded.
```

/clear

```
# 새 작업 시작
> 이제 user/profile 페이지에 아바타 업로드 기능을 추가하고 싶어.
[Claude는 이전 작업 맥락 없이 깨끗한 상태로 시작]
```

```
# 비교 - /clear 없이 계속하면
> 디버깅이 완료되었습니다.
> 이제 아바타 업로드 기능을 추가해 줘.
[Claude는 디버깅 컨텍스트가 남아 혼란스러울 수 있음]
```

/clear

완전 초기화

CLAUDE.md

자동 재로드

Fresh

새 작업 시작

Cost-save

토큰 절약

시나리오 1 / 디버깅 (1)

문제 정의 단계

SITUATION

운영 환경에서 사용자 프로필 페이지가 500 에러를 반환. 에러는 일부 사용자만 발생하며, 로그에는 부분적 스택트레이스만 있음.

첫 메시지 - 맥락과 증상을 정확히 전달
> 운영에서 다음 에러가 발생합니다. 원인 추적과 수정 부탁드립니다.

문제 정의

```
Error: Cannot read property 'preferences' of null
    at UserProfile.render (src/pages/user/profile.tsx:78)
    at processChild (next/dist/...)
```

영향: 사용자 약 5% (전체 1만명 중 500명)
재현: profile 페이지에서 일부 사용자만 발생

문맥:

- 어제 user 모델에 'preferences' jsonb 컬럼 추가
- 기존 사용자에게 마이그레이션은 미수행
- 이 컬럼이 null인 사용자들에서 발생할 가능성

Stack trace

스택트레이스 제공

Impact

영향 범위 명시

Context

최근 변경 컨텍스트

Hypothesis

추측 함께 제공

시나리오 1 / 디버깅 (2)

Claude의 자동 작업 흐름

STEP 01

Read
해당 파일

STEP 02

Grep
관련 호출

STEP 03

Read
DB 마이그레이션

STEP 04

Plan
수정안 제시

STEP 05

Edit
+ Bash 테스트

Claude의 작업 진행 (사용자 승인 없이 Read는 자동 실행):

Claude analysis

- Read src/pages/user/profile.tsx (line 70-90)
→ preferences 점(.) 접근 (null 체크 없음)
- Grep "user.preferences" → 3 callsites 발견
- Read prisma/migrations/..._add_preferences.sql
→ DEFAULT 값 없이 ALTER ADD COLUMN
- Read src/lib/userService.ts → fallback 없음

Plan: 옵셔널 체이닝 + fallback + 데이터 마이그레이션

Multi-Read

관련 파일 자동 식별

Grep

영향 범위 확인

Plan

수정 전 사용자 승인

Migration

코드+데이터 함께

시나리오 1 / 디버깅 (3)

도구 호출 시퀀스와 결과

tool execution

사용자가 Plan을 승인한 후

```
[Edit] src/pages/user/profile.tsx
- <p>{user.preferences.theme}</p>
+ <p>{user.preferences?.theme ?? 'light'}</p>
```

```
[Edit] src/lib/userService.ts
+ const normalize = (u: User) => ({
+   ...u,
+   preferences: u.preferences ?? {},
+ })
...
- return user
+ return normalize(user)
```

```
[Write] prisma/migrations/2026_05_24_backfill_preferences.sql
+ UPDATE users SET preferences = '{}::jsonb WHERE preferences IS NULL;
```

```
[Bash] pnpm test -- user/profile
✓ 23 tests passed
```

```
[Bash] git diff --stat
3 files changed, 12 insertions(+), 4 deletions(-)
```

Edit

정밀 수정

Write

신규 파일 생성

Bash

테스트 자동 실행

Diff

변경 요약

시나리오 2 / 기능 추가 (1)

요구사항 전달

FEATURE REQUEST

"GET /api/users 엔드포인트에 cursor 기반 페이지네이션을 추가. 기본 페이지 크기 20개, 최대 100개. 응답에 next/prev 링크 헤더 포함."

> /api/users 엔드포인트에 cursor 기반 페이지네이션을 추가해 주세요.

feature spec

요구사항:

- cursor 파라미터: 마지막 본 항목의 id (base64 인코딩)
- limit 파라미터: 기본 20, 최대 100
- 응답 헤더 Link: <...?cursor=xxx>; rel="next"
- 정렬: created_at DESC, tiebreaker id DESC
- 호환성: 기존 호출자가 깨지지 않아야 함 (cursor 없으면 첫 페이지)

테스트도 함께 작성해 주세요.

스타일은 기존 컨벤션을 따라 주세요 (CLAUDE.md 참고).

Behavior

동작 명세

Compat

하위 호환 명시

Tests

테스트 동반 요청

CLAUDE.md

컨벤션 참조

시나리오 2 / 기능 추가 (2)

Plan 모드 활성화와 계획 검토

```
> Shift+Tab (Plan 모드 활성화)
[Plan mode: ON. Claude will plan first and ask for approval.]
```

```
Claude:
Plan for cursor-based pagination
```

```
Files to modify:
```

1. src/app/api/users/route.ts - 핸들러 수정
2. src/lib/pagination.ts - 신규 헬퍼 (encode/decode)
3. src/lib/users.ts - 쿼리 함수 시그니처 변경

```
Tests to add:
```

1. tests/api/users.pagination.test.ts
 - 기본 페이지 크기
 - limit 경계값 (0, 100, 101)
 - cursor 정확도
 - Link 헤더 형식
 - 빈 결과

```
Database
```

- 인덱스 점검: (created_at DESC, id DESC) 이미 존재
- 추가 마이그레이션 불필요

```
Approve? (y/n/edit)
```

Plan

Shift+Tab

Plan 모드 토글

Plan first

먼저 계획

Approve

명시적 승인

Edit

계획 수정 가능

시나리오 2 / 기능 추가 (3)

승인 후 자동 실행과 PR

execution

> y (Plan 승인)

```
[Write] src/lib/pagination.ts (39 lines)
[Edit] src/app/api/users/route.ts (cursor 처리 + Link header)
[Edit] src/lib/users.ts (시그니처 변경: paginated 옵션)
[Write] tests/api/users/pagination.test.ts (5 test cases)
[Bash] pnpm test -- pagination → 5/5 passed
[Bash] pnpm test → 142/142 passed (회귀 없음)
[Bash] pnpm lint → no issues
```

Git

```
[Bash] git checkout -b feat/users-pagination
[Bash] git add ...
[Bash] git commit -m "feat(api): cursor-based pagination on /users"
[Bash] gh pr create --title "..." --body "..."
> Created PR #234: https://github.com/.../pull/234
```

총 소요: 4분 23초 / 토큰: 18,742 / 비용: \$0.18

5 files

동시 작업

5 tests

신규 테스트

142 pass

전체 회귀 통과

4m23s

소요 시간

시나리오 3 / 리팩토링

대규모 코드 정리 작업

refactor

> 모든 API 핸들러에서 try/catch + console.error 패턴을
중앙 errorHandler 미들웨어로 통합해 주세요.

목표: 47개 파일, 일관된 에러 로깅, Sentry 통합

Claude:

Grep "console.error" --include "*.ts" → 47 matches

Read src/middleware/errorHandler.ts → 이미 존재, Sentry 통합 완료

Plan

47개 파일에서 try/catch + console.error 패턴 제거

errorHandler 미들웨어를 라우터 레벨에 적용

개별 핸들러는 throw만 (catch는 미들웨어 담당)

예상 변경: ~200 lines 삭제, ~50 lines 추가

테스트: 기존 통과 + 새 시나리오 추가

Approve? (y/n/edit/preview)

47 files

영향 범위

Centralize

중앙화 패턴

-200/+50

코드 감소

Sentry

관측 통합

Plan 모드 vs Auto 모드

언제 어느 모드를 쓸 것인가

PLAN MODE

계획 후 실행

1. 큰 변경, 다중 파일 작업
2. 의사 결정이 중요한 작업
3. 신중함 우선
4. 사용자 합의 후 진행
5. 잘못된 방향 조기 차단

AUTO MODE

바로 실행

1. 작고 명확한 변경
2. 일상적 반복 작업
3. 속도 우선
4. 도구 호출별 승인 모드
5. 빠른 피드백 루프

WHEN TO SWITCH

아키텍처 변경, 데이터 마이그레이션, 50+ 파일 변경 → Plan 모드. 소규모 버그 수정, 단일 함수 작성 → Auto 모드.

Shift+Tab

모드 토글

Big

Plan 적합

Small

Auto 적합

Mix

세션 중 전환 가능

Plan 모드 활성화 상세

키보드와 명령행 옵션

```
# 1. REPL에서 토클 (가장 일반적)
> [Shift + Tab]
> [Plan mode: ON] (오렌지색 표시)

# 다시 누르면 OFF
> [Shift + Tab]
> [Plan mode: OFF]

# 2. 명령행 플래그로 시작 시 강제
$ claude --plan-mode
# 또는
$ claude -p "리팩토링해" --plan-mode

# 3. settings.json으로 기본값 설정
{
  "defaultMode": "plan"
}

# 4. Plan 모드에서 사용 가능한 추가 명령
> /plan      # 현재까지의 계획만 보기
> /accept    # 마지막 계획 승인
> /reject    # 계획 거부 후 재논의
```

plan mode

Shift+Tab

런타임 토클

--plan-mode

CLI 강제

defaultMode

설정 기본값

/accept

계획 승인

세션 관리 / /resume

특정 세션 ID로 재개

resume

```
# 1. 사용 가능한 세션 목록
$ claude --list-sessions
sess_abc123 | ~/projects/my-app | 2026-05-24 10:30 | 23 messages
sess_def456 | ~/projects/api-svc | 2026-05-23 18:15 | 41 messages
sess_ghi789 | ~/projects/my-app | 2026-05-23 09:00 | 89 messages

# 2. 특정 세션 재개
$ claude -r sess_abc123
> [Resumed session sess_abc123]
> [Loaded 23 messages, context 24% used]

# 3. REPL 내부에서
> /resume sess_abc123

# 4. 세션 검색
$ claude --search-sessions "pagination"
sess_abc123 | matched: "cursor-based pagination", "Link header"

# 5. 세션 삭제 (오래된 정리)
$ claude --delete-session sess_ghi789
```

-r <id>

세션 재개

--list

세션 목록

--search

키워드 검색

--delete

세션 삭제

세션 관리 / --continue

직전 세션 빠르게 이어가기

continue

```
# 1. 가장 최근 세션 이어가기
$ claude -c
> [Continuing last session sess_abc123]
> [Loaded 23 messages]

# 또는 긴 형식
$ claude --continue

# 2. 디렉토리별 마지막 세션 자동 인식
$ cd ~/projects/my-app
$ claude -c
> [Continuing last session in this directory]

$ cd ~/projects/api-svc
$ claude -c
> [Continuing last session in this directory]

# 3. --continue와 --resume의 차이
# --continue (-c): 같은 디렉토리의 가장 최근 세션
# --resume (-r): 세션 ID 명시적 지정
```

-c

단축 플래그

cwd

디렉토리별 자동 식별

Last

가장 최근만

vs -r

명시적 ID 필요 시 -r

Part 5 Recap

Quick Start 핵심 정리

TAKEAWAY 1

명확한 프롬프트

맥락, 목표, 제약, 검증 방법
4단계 구조로 작성.

TAKEAWAY 2

슬래시 명령 8개

help, init, status, clear,
compact, model, cost,
login 8개 필수.

TAKEAWAY 3

Plan vs Auto

큰 변경은 Plan, 작은 작업
은 Auto. Shift+Tab으로
전환.

TAKEAWAY 4

세션 관리

-c로 빠른 이어가기, -r로 명
시적 세션 전환, --list로 검
색.

NEXT - PART 6

Part 6은 Core Capabilities입니다.

Read, Edit, Bash, Grep, Glob, WebSearch 등 모든 내장 도구를 28장에 걸쳐 자세히 다룹니다.

Part 5

완료

Part 6

Core Capabilities (28p)

Part 7

IDE Integration (14p)

~2h

잔여 시간

CHAPTER 01 - Part 06

Overview – Core Capabilities

Claude Code의 정체성, 아키텍처, 설치, 인증, 핵심 기능, 워크플로 패턴까지



Part 6 / Core Capabilities

내장 도구를 모두 다루는 마스터 클래스

PART OBJECTIVE

Read, Edit, Bash, Grep 등 모든 내장 도구를 깊이 다루고, 권한 모델과 안전성 제어까지 28장에 걸쳐 학습합니다.

FILES

Read/Edit/Write

파일 입출력 도구의 정밀 사용법과 멀티모달 입력을 다룹니다.

SHELL

Bash 실행

동기/백그라운드 실행, 환경 격리, 타임아웃 같은 안전 옵션.

SEARCH

Grep/Glob/Web

정규식 검색, 파일 패턴, 웹 검색/폐치까지 정보 수집 도구.

GUARD

Permission

allowed/denied/ask, Auto-accept, Sandbox 안전 모델 전체.

28 Slides

Part 6 분량

~50 min

예상 강의 시간

11+

도구 종류

5 Modes

권한 모드

Read / 파일 읽기

텍스트 파일의 표준 읽기 도구

Read tool

```
# Read 도구 - Claude가 자동으로 호출
Read(file_path: "src/services/payment.js")

# 줄 범위 지정 (큰 파일)
Read(file_path: "src/app.js", offset: 100, limit: 50)
# → 100번째 줄부터 50줄 읽기

# 기본 동작
- 절대 경로 권장 (상대 경로도 지원)
- 자동 줄 번호 표시 (cat -n 형식)
- 큰 파일: 기본 2,000줄까지 읽음 (offset/limit로 조정)
- 빈 파일/존재하지 않는 파일: 명시적 오류 반환

# 사용 시점
- 파일 수정 전 반드시 Read로 현재 내용 확인 (필수)
- Grep으로 찾은 위치를 정확히 읽을 때
- README/설정 파일/스택 트레이스 파일 분석 시
```

cat -n

줄 번호 형식

2000

기본 라인 수

offset/limit

범위 지정

Required

Edit 전 필수

Read / 멀티모달 입력

이미지와 PDF 직접 읽기

IMAGE

이미지 파일

PNG, JPG, GIF, WebP 등을 직접 인식.
. UI 스크린샷에서 텍스트와 레이아웃 분석 가능.

PDF

PDF 문서

PDF 텍스트와 페이지 레이아웃을 동시에 분석. 사양서, 설계서 분석에 적합.

NOTEBOOK

Jupyter (.ipynb)

셀 단위로 코드와 출력을 모두 인식.
ML/DS 워크플로 통합 가능.

사용 예시
User: design.png을 보고 React 컴포넌트로 구현해 주세요.
Claude: [Read(design.png) → 이미지 분석] → 컴포넌트 생성

User: spec.pdf의 3페이지 요구사항을 코드로 옮겨 주세요.
Claude: [Read(spec.pdf) → PDF 읽기] → 해당 페이지 분석 → 구현

examples

PNG/JPG

이미지 인식

PDF

문서 분석

.ipynb

Jupyter 인식

Vision

Claude 비전 활용

Read / Line Range 활용

큰 파일을 효율적으로 다루기

Large file workflow

대용량 파일 처리 패턴

1. 전체 줄 수 확인 (Bash로)

```
wc -l src/app.js
```

→ 5234

2. 단계별 읽기 (Claude가 자동 수행)

```
Read(file_path: "src/app.js", offset: 1, limit: 100)
```

→ 헤더, import, 전역 변수 확인

```
Read(file_path: "src/app.js", offset: 1500, limit: 200)
```

→ 특정 함수 영역 정밀 분석

3. Grep과 조합 (가장 효율적)

```
Grep(pattern: "function processPayment", path: "src/")
```

→ src/app.js:1234

```
Read(file_path: "src/app.js", offset: 1230, limit: 80)
```

5000+
대용량 기준

offset
시작 라인

limit
읽을 줄 수

Grep+Read
효율 조합

Edit / Find & Replace

정밀한 부분 수정 도구

Edit tool

```
# Edit 도구의 기본 형태
Edit(
  file_path: "src/services/user.js",
  old_str: "return user;",
  new_str: "if (!user) throw new NotFoundError();\n return user;"
)

# 핵심 규칙
- old_str은 파일에서 유일해야 함 (중복 시 에러)
- 들여쓰기까지 정확히 일치해야 함
- 충분한 컨텍스트 포함 권장 (앞뒤 2-3줄)
- replace_all: true 옵션으로 모든 일치 수정 가능

# 잘못된 예
Edit(old_str: "return user", new_str: "...")
# → 같은 줄이 여러 곳에 있다면 에러

# 올바른 예
Edit(
  old_str: "  if (id) {\n    user = await User.find(id);\n    return user;\n  }",
  new_str: "..."
)
```

Unique

old_str 유일성 필수

Exact

들여쓰기 정확 매칭

replace_all

전체 치환 옵션

Atomic

원자적 적용

Edit / 멀티 파일 동시 수정

대규모 리팩토링 패턴

Multi-file refactor

Claude는 여러 파일을 순차적으로 또는 병렬로 수정 가능

User: 모든 컴포넌트의 default export를 named export로 바꿔 주세요.

Claude:

1. Grep "export default" → 23개 파일 식별
2. Read 각 파일 → 컴포넌트 이름 확인
3. Edit 각 파일 (병렬 실행 가능)
 - export default Button → export { Button }
 - export default Card → export { Card }
 - ...
4. Grep "import .* from" → 호출자 47곳 식별
5. Edit 47개 import 문 수정
 - import Button from → import { Button } from
6. Bash npm run lint && npm test

Sequential

순차 수정

Parallel

병렬 수정 가능

Atomic

파일별 원자성

Verify

수정 후 lint/test

Write / 신규 파일 생성

존재하지 않는 파일 만들기

Write tool

```
# Write 도구
Write(
  file_path: "src/lib/logger.ts",
  content: `import pino from 'pino';

export const logger = pino({
  level: process.env.LOG_LEVEL || 'info',
  redact: ['password', 'token', 'apiKey'],
});

export type Logger = typeof logger;`
)

# 주요 규칙
- file_path는 존재하지 않아야 함 (있으면 Edit 사용)
- 부모 디렉토리는 자동 생성됨
- 줄 끝 개행 자동 처리
- 파일 권한은 기본 644 (실행파일은 chmod 별도)

# Write vs Edit
- Write: 새 파일 생성 전용
- Edit: 기존 파일 수정 (Read 후 사용)
```

New only

신규 파일 전용

mkdir -p

부모 디렉토리 자동

0644

기본 권한

chmod

실행파일은 별도

NotebookEdit / Jupyter

ML/DS 워크플로 통합

NotebookEdit

```
# NotebookEdit 도구
NotebookEdit(
    notebook_path: "analysis/sales_q4.ipynb",
    cell_id: "abc-123",      # 또는 새 셀 추가
    new_source: "import pandas as pd\nndf = pd.read_csv(...)",
    cell_type: "code",      # code | markdown
    edit_mode: "replace",   # replace | insert | delete
)

# 셀 추가
NotebookEdit(
    notebook_path: "...",
    cell_id: "abc-123",     # 이 셀 다음에 삽입
    new_source: "df.describe()",
    cell_type: "code",
    edit_mode: "insert",
)

# 활용 패턴
- EDA 워크플로 자동화
- 분석 결과 보고서 작성
- ML 실험 추적
```

.ipynb

Jupyter 노트북

replace

셀 교체

insert

셀 추가

delete

셀 삭제

Bash / 명령 실행

셸 명령 동기 실행

Bash sync

```
# Bash 도구 - 동기 실행 기본
Bash(
  command: "npm test -- user.test.js",
  description: "사용자 테스트 실행", # 필수
  timeout: 60000, # ms, 기본 120초, 최대 600초
)
```

```
# 다중 명령은 && / || / ; 로
Bash(command: "cd src && npm install && npm run build")
```

표준 출력과 표준 에러 모두 수집

- stdout: 명령 정상 출력
- stderr: 에러 출력
- exit_code: 0이 정상, 비0이 실패

안전 권장사항

- 절대 경로 사용 권장 (cd 의존도 줄이기)
- 위험 명령 (rm -rf, sudo, curl|sh) 자동 차단됨
- 파일 변경은 Bash 대신 Edit/Write 사용

120s

기본 timeout

600s

최대 timeout

stdout+err

출력 모두 수집

description

필수 인자

Bash / 백그라운드 실행

장시간 실행 작업의 분리

Bash async

```
# 백그라운드 실행 - run_in_background: true
Bash(
  command: "npm run dev",
  description: "개발 서버 시작",
  run_in_background: true,
)
# → shell_id 반환, 즉시 다음 작업 계속

# 출력 확인 - BashOutput 도구
BashOutput(bash_id: "shell-123", filter: "ERROR")
# → 누적된 출력을 가져옴

# 종료 - KillShell 도구
KillShell(shell_id: "shell-123")

# 활용 패턴
- 개발 서버 (npm run dev / docker compose up)
- 파일 워치 (vite, webpack)
- 로그 tail 모니터링
- 장시간 빌드 작업
```

run_in_background

true 설정

shell_id

핸들 반환

BashOutput

출력 폴링

KillShell

종료

Bash / 환경 격리

sandbox와 작업 디렉토리

CWD

작업 디렉토리

Claude Code 시작 시 cwd가 모든 Bash의 기본 위치.
cd로 변경한 디렉토리는 다음 Bash 호출에서 유지되지 않음.

ENV

환경 변수

시작 시점의 환경변수가 모든 명령에 전달됨.
추가 변수는 inline 또는 export 사용.

TIMEOUT

Timeout 강제

120초 기본, 최대 600초.
무한 루프나 hang을 방지하는 안전 장치.

KILL

강제 종료

KillShell 또는 사용자 Ctrl+C로 즉시 종료.
자식 프로세스도 함께 종료.

NOTE

Claude Code는 격리된 컨테이너가 아닌 사용자 셸을 사용합니다. 위험한 작업은 컨테이너/VM 환경에서 실행하는 것을 권장합니다.

cwd

시작 디렉토리 고정

env

환경변수 상속

120/600s

Timeout 기본/최대

Ctrl+C

강제 중단

Grep / 패턴 검색

코드베이스 전체 텍스트 검색

Grep tool

```
# Grep 도구 - ripgrep 기반
Grep(
  pattern: "function processPayment",
  path: "src/",          # 검색 디렉토리
  glob: "*.ts",          # 파일 필터 (선택)
  type: "ts",            # 또는 언어 타입
  output_mode: "files_with_matches", # 기본
  -i: true,              # 대소문자 무시
  -n: true,              # 라인 번호 표시
  -A: 3, -B: 2,          # 매칭 앞뒤 라인 포함
)

# 출력 모드
- files_with_matches: 매칭된 파일만
- content: 매칭된 라인 내용
- count: 매칭 개수

# 활용 예시
- 함수/변수 정의 위치 찾기
- TODO/FIXME 일괄 수집
- 사용 위치(call site) 분석
```

ripgrep

내부 구현

PCRE

정규식 엔진

gitignore

자동 인식

fast

대용량 친화

Grep / 정규식 활용

복잡한 패턴 매칭

Regex patterns

자주 쓰는 정규식 패턴

1. HTTP 메서드 호출 패턴

Grep(pattern: "(get|post|put|delete)\\(['\"]")

→ axios.get("..."), fetch("..."), etc.

2. 환경변수 사용 위치

Grep(pattern: "process\\.env\\.([A-Z_]+)")

→ process.env.API_KEY, process.env.DB_HOST

3. 비밀 키 패턴 (보안 점검)

Grep(pattern: "(sk-|api[_-]?key|secret)['\"]?[:=]['\"]")

→ 하드코딩된 비밀 발견

4. TODO/FIXME 분류

Grep(pattern: "(TODO|FIXME|HACK|XXX):", output_mode: "content")

5. 함수 시그니처 검색

Grep(pattern: "^\\(export\\s+\\s+\\)?async\\s+function\\s+\\s+\\w+",
glob: "*.ts")

PCRE

Perl 호환 정규식

\\w\\W

백슬래시 이스케이프

^ \$ \\w\\Wb

anchors

() | *

Groups

Glob / 파일 패턴 매칭

파일 경로 기반 검색

Glob tool

```
# Glob 도구 - 파일 경로 패턴 매칭
Glob(pattern: "**/*.test.ts")
# → 모든 *.test.ts 파일 (재귀)

Glob(pattern: "src/**/*.tsx")
# → src 아래 components 폴더의 tsx 파일

# 자주 쓰는 패턴
**/*.{js,ts,jsx,tsx}    # JS/TS 전체
src/**/*.index.ts      # 모든 index.ts
**/*.{test,spec}.*      # 테스트 파일 전체
docs/**/*.md            # 문서 파일
**/!(node_modules)/*    # node_modules 제외

# 활용 패턴
- 마이그레이션 대상 파일 식별
- 테스트 커버리지 범위 계산
- 빌드 산출물 정리
- 파일 명명 규칙 일관성 점검
```

**

재귀 매칭

*

단일 디렉토리

{,}

대체 매칭

?

단일 문자

WebSearch / 외부 검색

코드 외부의 최신 정보 확보

WebSearch

```
# WebSearch 도구
WebSearch(query: "React 19 useTransition 변경점")

# 활용 예시
User: 최신 Stripe API의 metered billing 변경사항 알려 주세요.
Claude: [WebSearch("Stripe metered billing 2026 changes")]
        → 검색 결과 요약 + 관련 URL 제시
        [WebFetch("https://stripe.com/docs/...")]
        → 페이지 상세 분석

# 사용 시점
- 라이브러리 최신 버전 변경사항
- 보안 취약점 정보 (CVE)
- 베스트 프랙티스 최신 동향
- API/SDK 공식 문서 참조

# 권장 패턴
- WebSearch로 URL 식별
- 그 후 WebFetch로 정밀 분석
- 결과를 코드에 반영하기 전 사용자에게 확인
```

Query

자연어 검색어

URL

결과로 URL 제공

+Fetch

Fetch와 조합

Live

실시간 정보

WebFetch / URL 가져오기

특정 페이지 상세 분석

WebFetch

```
# WebFetch 도구
WebFetch(
  url: "https://docs.anthropic.com/claude-code/overview",
  prompt: "Subagents 기능에 대한 핵심 요약과 사용 예시 추출",
)
```

```
# 동작
- 페이지 다운로드 + HTML→Markdown 변환
- prompt에 따라 Claude가 의미 추출 (LLM 단계)
- 결과: 구조화된 요약 + 핵심 인용
```

```
# 캐시 정책
- 동일 URL 15분간 캐싱 (네트워크 절약)
- prompt가 달라도 캐시 활용
- 강제 갱신 옵션은 별도
```

```
# 활용 예시
- 공식 문서에서 특정 섹션 추출
- 변경 로그에서 breaking change 식별
- GitHub PR/Issue 분석
```

HTML→MD

자동 변환

LLM

의미 추출 단계

15min

캐시 시간

Inline

결과 직접 활용

WebFetch / 캐싱과 최적화

효율적인 외부 정보 수집

CACHE

자체 캐시

동일 URL은 15분 캐싱. 같은 세션 내 반복 호출 시 네트워크 절약.

REDIRECT

리다이렉트 처리

301/302는 안전 도메인 검증 후 자동 추적. 외부 호스트 변경은 차단.

TIMEOUT

타임아웃

기본 30초. 응답 없는 페이지는 안전하게 실패 처리.

AUTH

인증 페이지

로그인 필요 페이지는 가져올 수 없음. 공개 페이지만 지원.

BEST PRACTICE

한 세션에서 같은 문서를 반복 참조할 때 캐시 활용으로 비용/속도 모두 이득. 보안상 인증 페이지는 명시적으로 거부.

15min

캐시 TTL

Public

공개 페이지만

30s

기본 타임아웃

Safe

리다이렉트 검증

Git 워크플로 / 일반

Bash를 통한 Git 통합

Git basics

```
# Claude Code는 Git 명령을 자유롭게 활용

# 상태 확인 (자주 사용됨)
Bash("git status")
Bash("git diff")
Bash("git log --oneline -20")

# 브랜치 생성/전환
Bash("git checkout -b feat/user-pagination")
Bash("git switch main")

# 변경사항 스테이징 / 커밋
Bash("git add src/users/")
Bash("git commit -m '...', ...) # 메시지 자동 작성

# 푸시
Bash("git push -u origin feat/user-pagination")

# 권한 정책 예시 (settings.json)
"allow": ["Bash(git status)", "Bash(git diff:*)",
          "Bash(git log:*)", "Bash(git branch:*)"],
"ask":   ["Bash(git push:*)", "Bash(git merge:*)"]
```

git status

자주 자동 호출

git diff

변경 확인

commit

메시지 자동 작성

push

권장: ask 정책

Git / 커밋 메시지 자동 작성

Claude가 작성하는 의미 있는 메시지

Commit message

```
# Claude의 커밋 메시지 작성 패턴

# 1. 변경 분석
git diff --staged
# → 어떤 파일이 어떻게 바뀌었는지 분석

# 2. Conventional Commits 형식 생성
feat(users): add pagination with cursor-based navigation

- Add page/limit query parameters with sensible defaults
- Implement Link header for next/prev navigation
- Update tests for new pagination behavior

# 3. 커밋 실행
git commit -m "$(cat <<'EOF'
feat(users): add pagination with cursor-based navigation
...
EOF
)"

# 권장: Co-authored-by Claude 추가 (투명성)
# Co-Authored-By: Claude <noreply@anthropic.com>
```

Conventional

표준 형식

feat/fix/...

Type prefix

scope

범위 명시

Co-Author

Claude 표시

Git / PR 생성

gh CLI를 통한 자동 PR

PR creation

```
# 1. gh CLI 인증 (사전 1회)
gh auth status
# 또는: gh auth login

# 2. Claude가 PR을 생성하는 흐름
Bash("git push -u origin feat/user-pagination")

Bash("gh pr create --title '...' --body \"$(cat <<'EOF'
## Summary
사용자 API에 페이지네이션 추가.

## Changes
- Cursor 기반 페이지네이션 구현
- Link 헤더로 next/prev 이동 지원
- 단위 테스트 12개 추가

## Test Plan
- npm test -- pagination (passes ✓)
- 수동 검증: /users?page=2&limit=20

🤖 Generated with Claude Code
EOF
)\")")
```

gh CLI
GitHub 공식

--title
제목 자동 작성

--body
본문 자동 작성



AI 작성 표시

GitHub / gh CLI 통합

PR 외 다양한 GitHub 작업

PR REVIEW

PR 분석

gh pr view, gh pr diff로 PR을 가져와 Claude가 리뷰 코멘트를 자동 생성.

ISSUES

Issue 처리

gh issue list로 할당된 이슈를 가져와 분석하고, 코드 변경 시 자동 연결.

ACTIONS

Workflow

gh run list, gh run view로 CI 실패 분석. 로그 자동 검토.

RELEASE

Release Notes

gh release create로 태그 + 릴리즈 노트 자동 생성. 변경 로그 자동 작성.

POWER PATTERN

Claude와 gh CLI 조합으로 GitHub의 거의 모든 작업을 자동화할 수 있음. CI 실패에서 로그 분석, 수정 PR까지 한 흐름으로 가능.

pr

PR 명령군

issue

Issue 명령군

run

Actions 명령군

release

Release 명령군

Permission 모델 / 개요

안전한 자율 실행의 핵심

ALLOW

자동 허용

Read, git status 등 안전한 도구는 사용자 승인 없이 자동 실행.

ASK

사용자 확인

Edit, Write 등 변경 작업은 매번 사용자 확인을 받음. 기본값.

DENY

명시적 거부

rm -rf, sudo, curl|sh 같은 위험 명령은 시도 자체를 차단.

PRECEDENCE

Deny가 가장 강력. 같은 명령에 대해 allow와 deny가 충돌하면 deny가 우선. 패턴 매칭은 그룹 형식.

allow

자동 허용 패턴

ask

사용자 확인 패턴

deny

거부 패턴 (최우선)

Glob

패턴 매칭 형식

allowedTools / deniedTools

실전 패턴 모음

```
# settings.json 또는 --allowed-tools 플래그
{
  "permissions": {
    "allow": [
      "Read(**)",           # 모든 파일 읽기
      "Glob(**)", "Grep(**)", # 모든 검색
      "Bash(git status)",   # 정확 매칭
      "Bash(git diff:*)",   # git diff 시작
      "Bash(git log:*)",
      "Bash(npm test:*)",
      "Bash(npm run test:*)",
      "Bash(node --version)",
      "WebSearch(**)",
    ],
    "ask": [
      "Edit(**)", "Write(**)",
      "Bash(git push:*)",
      "Bash(git commit:*)",
      "Bash(npm install:*)",
    ],
    "deny": [
      "Bash(rm -rf:*)",
      "Bash(sudo:*)",
      "Bash(curl * | sh)",
      "Bash(* | bash)",
      "Edit(.env)", "Edit(secrets/**)"
    ]
  }
}
```

permissions

Read()**

모든 읽기 허용

:*

명령 시작 매칭

.env

시크릿 파일 보호

deny first

거부 최우선

Ask 단계별 승인

대화형 모드의 기본 동작

Ask UX

사용자 경험 - 매 위험 작업마다 확인 받음

Claude: I want to modify src/services/payment.js:

```
- return amount;  
+ return amount * (1 + taxRate);
```

Approve this change? [y/N/q/a/d]

y - yes, apply this single change

N - no, skip and continue

q - quit, cancel rest of work

a - yes, and auto-approve similar future Edits

d - drop into debug, show me the file context

그룹 승인

- 같은 도구의 연속 호출은 한 번에 승인 가능

- 'a' 응답으로 세션 동안 자동 승인

안전 장치

- 무한 자동 동의 방지 (n번 후 재확인)

- 사용자 abort 시 즉시 중단

[y/N/q/a/d]

5가지 응답

Default

기본 N (안전)

'a' key

유사 작업 자동

Abort

q로 즉시 중단

Auto-accept 모드

위험한 명령 외 자동 진행

auto-accept

```
# 활성화 방법
# 1. 대화형 세션에서 Shift+Tab으로 모드 토글
# 2. 또는 --permission-mode auto-accept 플래그

# Auto-accept 동작
- ask 목록의 도구도 자동 진행 (사용자 확인 없이)
- deny 목록은 여전히 거부됨
- 변경은 모두 적용되지만 사용자가 추후 git diff로 검토 가능

# 활용 시점
- 신뢰할 수 있는 작업 (예: 단순 리팩토링)
- 대량의 반복 변경 처리
- 사용자가 옆에서 모니터링 중인 경우

# 주의사항
- 한밤중 대량 실행은 피하기 (검토 없이 누적되면 위험)
- Git을 통한 변경 가시화 필수
- Yolo 모드와는 다름 (deny 여전히 동작)
```

Shift+Tab

대화형 토글

--auto

CLI 플래그

deny 유지

Yolo와 차이점

git diff

추후 검토 필수

Safe Yolo Mode

⚠️ 모든 권한 일괄 허용 (위험)

⚠️ DANGEROUS

```
# 활성화 (CLI 플래그)
claude --dangerously-skip-permissions

# 또는 환경변수
export CLAUDE_DANGEROUSLY_SKIP_PERMISSIONS=true

# Yolo 모드 동작
- allow/ask/deny 정책 모두 무시
- 모든 도구 호출이 즉시 실행
- 사용자 확인 없음

# 절대 권장하지 않는 경우
- 프로덕션 시스템에 직접 연결된 환경
- 자격증명이 있는 머신
- 검토 없이 대량 변경이 누적될 수 있는 환경

# 적절한 사용 사례 (제한적)
- 격리된 컨테이너/VM 안
- 일회성 자동화 작업
- 충분히 검증된 작업 + 사용자 모니터링
- 단, 항상 Git으로 변경 추적 가능해야 함
```

dangerous

플래그 이름의 경고

No deny

deny조차 무시

Container

격리 환경 한정

Git

추적 가능 필수

Sandbox / Safety 가드레일

엔터프라이즈 환경의 다층 안전 장치

CONTAINER

격리 컨테이너

Docker/Podman 안에서
실행해 호스트 파일 시스템
격리. 실패해도 영향 한정.

HOOKS

Pre/Post Hooks

도구 호출 전후로 사용자 정
의 스크립트 실행.
추가 검증/감사 가능.

POLICY

엔터프라이즈 정책

IT 관리자가 settings를 강
제 배포.
사용자가 deny 정책을 악화
하지 못함.

AUDIT

감사 로그

모든 도구 호출이
OTel/SIEM으로 전송.
사후 추적과 컴플라이언스.

DEFENSE IN DEPTH

한 계층이 우회되어도 다른 계층이 차단. Yolo 모드를 사용하더라도 컨테이너 격리와 감사 로그가 안전망이 됨.

Container

실행 격리

Hooks

커스텀 검증

Enterprise

강제 정책

Audit

로그/SIEM

Part 6 Recap

Core Capabilities 정리

TAKEAWAY 1

11+ 내장 도구

Read/Edit/Write/Bash/
Grep/Glob/Web 등 도구
의 정확한 사용법.

TAKEAWAY 2

Git 깊이 통합

커밋 메시지, PR 생성, gh
CLI까지 자연어 한 줄로 자
동화.

TAKEAWAY 3

권한 모델

allow/ask/deny 3단계와
Auto-accept, Yolo,
Sandbox 모드.

TAKEAWAY 4

Defense in Depth

컨테이너 + Hooks + 엔터
프라이즈 정책 + 감사 로그
다층 안전.

NEXT - PART 7

Part 7은 IDE Integration. VS Code와 JetBrains 플러그인 14장에 걸쳐 다룹니다.

Part 6

완료

Part 7

IDE (14p)

Part 8

CLAUDE.md (16p)

~1.5h

Chapter 1 잔여

CHAPTER 01- Part 07

Overview – IDE Integration

Claude Code의 정체성, 아키텍처, 설치, 인증, 핵심 기능, 워크플로 패턴까지



Part 7 / IDE Integration

VS Code와 JetBrains 플러그인 활용

PART OBJECTIVE

Claude Code의 IDE 플러그인을 활용해 코딩 워크플로 안에서 자연스럽게 Claude를 호출하고, 결과를 인라인으로 검토하는 방법을 학습합니다.

VS CODE

VS Code 확장

Marketplace 설치, 사이드바, Diff, 단축키 통합

JETBRAINS

JetBrains 플러그인

IntelliJ/PyCharm/WebStorm 등 모든 JB IDE 지원

EDITOR

Vim/Emacs

외부 에디터에서도 터미널 통합으로 활용 가능

SHORTCUTS

키보드 단축키

효율적 사용을 위한 단축키 종합 정리

14 Slides

Part 7 분량

~20 min

예상 강의 시간

2+IDEs

주요 통합 대상

Inline

Diff/Edit 통합

VS Code 확장 / 설치

Marketplace에서 빠른 시작

VS Code Install

```
# 1. VS Code Marketplace 검색
View → Extensions (Ctrl/Cmd+Shift+X)
Search: "Claude Code"
Publisher: Anthropic

# 2. 또는 CLI로 설치
code --install-extension anthropic.claude-code

# 3. 명령 팔레트 (Cmd/Ctrl+Shift+P)
> Claude Code: Open Sidebar
> Claude Code: Start Session
> Claude Code: Sign In

# 4. 인증
- Sign In 명령 실행
- 브라우저 OAuth 또는 API Key 입력

# 5. 사이드바 활성화
- Activity Bar의 Claude 아이콘 클릭
- 또는 Ctrl/Cmd+Shift+L
```

Marketplace

공식 게시

anthropic

Publisher

Cmd+Shift+L

사이드바 토클

OAuth/Key

인증 옵션

VS Code 확장 / 주요 기능

4대 핵심 기능

SIDEBAR

대화 사이드바

IDE 옆에 상시 열어두고 대화 가능.
파일 선택과 자동 연동.

DIFF

인라인 Diff

편집 제안을 IDE의 native diff 뷰로 표시.
라인별 승인 가능.

CONTEXT

컨텍스트 자동 전달

활성 파일, 선택 영역, 열린 탭이 자동으로 Claude에 전달됨.

TERMINAL

통합 터미널 연동

IDE 내장 터미널에서 claude를 실행하면 IDE 상태와 자동 연동.

PRODUCTIVITY

CLI보다 IDE 통합이 일상 작업에 더 자연스러우며, 다만 자동화는 여전히 CLI/SDK가 강점.

Native UI

익숙한 UX

Diff View

시각 검토

Auto Context

자동 컨텍스트

Terminal

터미널 연동

VS Code / claude code 명령

명령 팔레트에서 활용

commands

명령 팔레트 (Cmd/Ctrl+Shift+P) 주요 명령

> Claude Code: Open Sidebar	# 사이드바 토글
> Claude Code: Start New Session	# 새 세션
> Claude Code: Continue Last Session	# 마지막 세션 이어가기
> Claude Code: Send Current Selection	# 선택 영역 전송
> Claude Code: Send Current File	# 활성 파일 전송
> Claude Code: Toggle Plan Mode	# Plan 모드 토글
> Claude Code: Show Cost	# 사용량 표시
> Claude Code: Sign Out / Sign In	# 인증 관리
> Claude Code: Open Settings	# 확장 설정 열기

컨텍스트 메뉴 (편집기에서 우클릭)

- Ask Claude about this file
- Refactor selection with Claude
- Explain this code
- Generate tests for this function

> claude

검색 prefix

Selection

선택 영역 활용

Context Menu

우클릭 메뉴

Sign In/Out

계정 관리

VS Code / 선택 영역 컨텍스트

효율적인 정밀 작업

Selection workflow

선택 영역 전달 워크플로

1. 편집기에서 코드 블록 선택 (드래그 or Shift+화살표)
2. 우클릭 → "Ask Claude about this code"
또는 Cmd/Ctrl+Shift+P → "Send Current Selection"
3. 사이드바에 자연스럽게 컨텍스트 추가됨
4. 자연어 질문 입력

활용 예시

[선택: `function processPayment() { ... }`]
User: 이 함수의 에러 처리를 개선해 주세요.

또는 다중 영역

1. Cmd/Ctrl+Click로 여러 영역 선택
2. 모두 한 번에 컨텍스트로 전달
3. 함수 사이의 관계 분석 등 다중 영역 작업 가능

단축키

- Shift+Alt+M: 선택 영역 + 자연어 질문 (한 번에)
- Cmd+I: 선택 영역 인라인 편집 요청

Selection

드래그 선택

Multi

다중 영역

Shift+Alt+M

빠른 질문

Cmd+I

인라인 편집

VS Code / Diff 통합

편집 제안의 시각 검토

INLINE

인라인 Diff

편집 위치에 직접 Diff가 표시됨.
빨강/초록 색상으로 변경 확인.

SIDE-BY-SIDE

나란히 보기

Diff 뷰어로 좌우 비교.
큰 변경 검토에 적합.

LINE-LEVEL

라인별 승인

특정 라인만 선택 적용/거부 가능.
부분 적용 지원.

UNDO

되돌리기

적용 후에도 Cmd/Ctrl+Z로 모든 변경을 되돌릴 수 있음.

TIP

큰 변경은 항상 side-by-side 모드로 검토. 작은 변경은 인라인이 더 효율적.

Inline

작은 변경

Side-by-side

큰 변경

Per-line

라인별 적용

Cmd+Z

복원 가능

JetBrains 플러그인 / 설치

모든 JB IDE에서 동일하게 사용

JetBrains

```
# 1. 플러그인 저장소에서 검색
File → Settings → Plugins (또는 Preferences on macOS)
Marketplace 탭 → "Claude Code" 검색
Install → IDE 재시작
```

```
# 2. 지원 IDE (동일 플러그인)
- IntelliJ IDEA (Ultimate / Community)
- PyCharm (Pro / Community)
- WebStorm
- GoLand
- RubyMine
- PhpStorm
- CLion
- Rider
- DataGrip (제한적)
- Android Studio
```

```
# 3. 인증
Tools → Claude Code → Sign In
- 브라우저 OAuth 또는 API Key
```

```
# 4. Tool Window 활성화
View → Tool Windows → Claude Code (Alt+9 권장)
```

Marketplace

JetBrains 공식

10+ IDEs

지원 IDE 종류

Alt+9

Tool Window 단축

Same plugin

단일 플러그인

JetBrains 활용

IntelliJ 특화 기능

INSPECT

Code Inspection

IntelliJ의 코드 분석 결과를 Claude에 자동 전달해 수정 안 받기.

REFACTOR

Refactor 통합

Rename, Extract 같은 IntelliJ Refactor 후 Claude의 추가 정리 제안.

RUN

Run Config

테스트/디버그 실행 결과를 Claude에 자동 전달해 분석 요청.

VCS

VCS 통합

IntelliJ의 VCS 도구창과 연동되어 변경사항 분석/커밋 메시지 자동.

JB POWER

JetBrains IDE의 강력한 정적 분석과 Claude의 자연어 추론을 결합해 가장 풍부한 컨텍스트 작업이 가능.

Inspection

정적 분석 통합

Refactor

IDE 리팩터 후속

Run Output

실행 결과 분석

VCS

버전 관리 통합

JetBrains 단축키

빠른 작업 흐름

동작	단축키 (default)	용도
Tool Window 토글	Alt+9	사이드바 열기/닫기
선택 영역 전송	Shift+Cmd+L (mac), Ctrl+Shift+L	코드 → Claude
인라인 편집 요청	Cmd+WW (mac), Ctrl+WW	바로 위치에서 수정
새 세션 시작	Cmd+Shift+N → Claude: New Session	세션 초기화
Plan 모드 토글	Shift+Alt+P	자율성 모드 변경
도움말 / 명령 검색	Shift Shift → "claude"	Search Everywhere

Alt+9
Tool Window

Cmd+L
선택 전송

Cmd+WW
인라인 편집

Shift×2
Search 명령

외부 에디터 / Vim

터미널 우선 워크플로

Vim integration

```
# 1. tmux와 함께 사용 (권장)
# 첫 화면: Vim으로 코드 편집
# 두 번째 분할: claude 세션
```

```
tmux new -s coding
# Ctrl+B → % : 수평 분할
# 왼쪽: vim src/app.js
# 오른쪽: claude
```

```
# 2. Vim 플러그인 (선택)
# vim-claude (커뮤니티 플러그인)
" .vimrc
Plug 'username/vim-claude'
```

```
# 명령:
:ClaudeAsk "이 함수를 리팩토링해 주세요"
:ClaudeSelection " 선택 영역 → Claude
```

```
# 3. 키 매핑 권장
nnoremap <leader>cc :silent !claude -p "<C-r>"+<CR>
" Yank 후 leader+cc로 Claude 호출
```

tmux
권장 결합

:ClaudeAsk
플러그인 명령

<leader>cc
키 매핑

CLI Direct
직접 호출

외부 에디터 / Emacs & Neovim

플러그인 생태계

Emacs/Neovim

```
# Neovim - claude.nvim 플러그인 (커뮤니티)
-- ~/.config/nvim/lua/plugins/claude.lua
return {
  "anthropics/claude.nvim",
  config = function()
    require("claude").setup({
      model = "sonnet",
      keymaps = {
        ask = "<leader>ca",
        edit = "<leader>ce",
        explain = "<leader>cx",
      },
    })
  end,
}

# Emacs - claude.el (커뮤니티)
;; init.el
(use-package claude
  :ensure t
  :bind (("C-c C-a" . claude-ask)
        ("C-c C-e" . claude-edit-region)))

# 두 경우 모두 기본은 claude CLI 호출
# 따라서 CLI 설치만 되어 있으면 즉시 사용 가능
```

claude.nvim
Neovim 플러그인

claude.el
Emacs 패키지

CLI
기반 동일

Community
커뮤니티 메인테인

컨텍스트 공유 베스트 프랙티스

IDE/CLI 양쪽 활용 패턴

PROJECT

CLAUDE.md

프로젝트 루트의
CLAUDE.md를 통해 IDE와
CLI가 동일한 컨텍스트 공유.

SESSION

세션 호환성

IDE에서 시작한 세션을 CLI
로 이어가거나 그 반대도 가
능 (/resume).

SETTINGS

settings.json

권한, MCP, 모델 설정이
IDE/CLI 모두에 동일 적용.

HISTORY

히스토리 통합

~/.claude/sessions에 통
합 저장되어 어디서든 검색
가능.

TIP

IDE는 정밀 편집, CLI는 자동화/대량 작업, 양쪽을 동시에 활용하는 것이 가장 효율적.

CLAUDE.md

컨텍스트 공유 키

/resume

세션 이어가기

settings

단일 설정

Unified

통합 히스토리

키보드 단축키 종합

효율적 사용을 위한 핵심 정리

범주	VS Code	JetBrains
사이드바 토글	Cmd/Ctrl+Shift+L	Alt+9
선택 전송	Shift+Alt+M	Shift+Cmd/Ctrl+L
인라인 편집	Cmd+I	Cmd/Ctrl+WW
Plan 모드	Cmd/Ctrl+Shift+P → Toggle Plan	Shift+Alt+P
새 세션	Cmd/Ctrl+Shift+P → New Session	Cmd+Shift+N → New
명령 검색	Cmd/Ctrl+Shift+P	Shift Shift

IDE-native

각 IDE 관례 따름

Customize

재할당 가능

Palette

명령 팔레트 활용

Print

치트시트 권장

Part 7 Recap

IDE Integration 정리

TAKEAWAY 1

VS Code 통합

Marketplace 설치, 사이드바/Diff/컨텍스트 자동 전달.

TAKEAWAY 2

JetBrains 통합

모든 JB IDE에서 동일한 플러그인, 정적 분석과 결합.

TAKEAWAY 3

외부 에디터

Vim/Neovim/Emacs도 커뮤니티 플러그인 + CLI로 활용 가능.

TAKEAWAY 4

단일 컨텍스트

CLAUDE.md, settings, 세션이 IDE/CLI 양쪽에서 공유됨.

NEXT - PART 8

Part 8은 CLAUDE.md와 메모리 시스템. 16장에 걸쳐 영구 컨텍스트 관리 학습.

Part 7

완료

Part 8

Memory (16p)

Part 9

Workflow (18p)

~1h

Chapter 1 잔여

CHAPTER 01 - Part 08

Overview – CLAUDE.md & Memory

Claude Code의 정체성, 아키텍처, 설치, 인증, 핵심 기능, 워크플로 패턴까지



Part 8 / CLAUDE.md & Memory

영구 컨텍스트와 메모리 계층

PART OBJECTIVE

CLAUDE.md 파일과 메모리 계층 구조를 학습해, 매 세션마다 같은 컨텍스트를 자동 로드함으로써 일관된 작업 결과를 얻는 방법을 배웁니다.

TOPIC 1

메모리 계층

Enterprise, User,
Project, Local 4계층 메모
리 시스템 이해

TOPIC 2

CLAUDE.md 작성

/init, /memory, # 단축,
import 문법으로 효과적으
로 작성

TOPIC 3

베스트 프랙티스

구조, 분량, 업데이트 주기
등 운영 노하우

TOPIC 4

실전 예시

Python, Node.js, 모노레
포 등 실제 프로젝트 템플릿

16 Slides

Part 8 분량

~25 min

예상 강의 시간

4 Layers

메모리 계층

Lab 2

실습 포함

메모리 계층 구조

4단계 계층으로 컨텍스트 관리

MEMORY LAYERS

Priority Order (높을수록 우선):

1. Enterprise	/etc/claude/CLAUDE.md	(조직)
조직 차원의 정책, 보안 요건, 표준 도구		
2. User	~/.claude/CLAUDE.md	(개인)
개인 선호, 코드 스타일, 자주 쓰는 명령		
3. Project	./CLAUDE.md	(프로젝트)
프로젝트 구조, 빌드/테스트 명령, 컨벤션		
4. Local	./.claude/local.md	(개인 X 프로젝트)
.gitignore 처리, 로컬 개발자 비밀 설정		

병합: 4계층이 합쳐져 단일 컨텍스트로 Claude에 전달됨

4 Layers

계층 수

Merge

4계층 자동 병합

Enterprise

최우선 (조직 정책)

Local

Git 미포함

Project Memory / ./CLAUDE.md

프로젝트별 컨텍스트

```
# Project: my-app
```

Stack

- TypeScript 5.4 / React 18 / Vite 5
- Express 4 / Prisma / PostgreSQL 15
- Jest + React Testing Library

Commands

- 개발: pnpm dev
- 빌드: pnpm build
- 테스트: pnpm test (감시: pnpm test:watch)
- 린트: pnpm lint
- 마이그레이션: pnpm prisma migrate dev

Conventions

- 함수형 컴포넌트 + Hooks (Class 금지)
- 타입은 src/types/ 에 모음
- 신규 API는 src/server/routes/ 아래 추가
- 모든 PR에는 테스트 동반 필수

./CLAUDE.md

./CLAUDE.md

프로젝트 루트

Git

버전 관리 포함

Team

팀 전체 공유

Auto-load

세션 시작 시 자동

User Memory / ~/.claude/CLAUDE.md

개인 선호 영구 저장

Personal Preferences

Communication Style

- 한국어로 응답
- 코드 변경 전 항상 Plan 제시
- 코드 변경 후 핵심 변경점 3줄 요약

Code Style (개인 기본값)

- 들여쓰기: 스페이스 2칸
- 따옴표: 작은따옴표 (JS/TS)
- import 순서: 외부 → 내부 → 상대경로
- 트레일링 콤마: 항상

Frequently Used Commands

- 빠른 테스트: `jest --watch --testPathPattern`
- Git 정리: `git rebase -i HEAD~5`
- 로그 검색: `ag -i` (the silver searcher)

Tools

- 패키지: pnpm 선호 (npm/yarn 대신)

~/.claude/CLAUDE.md

~/.claude/

사용자 홈

All projects

모든 프로젝트 적용

Personal

개인 전용

Override

프로젝트가 우선

Local Memory / .claude/local.md

개인 X 프로젝트 비밀 설정

.claude/local.md

Local-only Memory (.gitignore 포함)

Local Dev Environment

- 데이터베이스: postgres://localhost:5433 (포트 충돌)
- API Key: 시크릿 매니저에서 자동 로드 → ENV 주입
- 사내 프록시: 사용 안함 (집에서 작업)

Personal Workarounds

- node_modules 손상 시: `rm -rf node_modules && npm i`
- VSCode TS 서버 멈춤 시: `Cmd+Shift+P` → Restart TS

TODO (개인용)

- @username 부분 리팩토링 시 함께 검토 부탁
- 새 컴포넌트 라이브러리 평가 중

.claude/

프로젝트 내 폴더

local.md

파일명

.gitignore

버전 관리 제외

Per-dev

개인 X 프로젝트

Enterprise Memory

조직 차원의 정책과 표준

/etc/claude/CLAUDE.md (시스템 전체 적용)

/etc/claude/CLAUDE.md

Company Policy

- 사내 코드를 외부에 절대 노출 금지
- 모든 commit message는 한국어 + 영어 병행
- AI 생성 코드는 PR description에 [AI-generated] 표기

Required Practices

- 모든 신규 API는 OpenAPI 스펙 포함
- DB 변경은 마이그레이션 파일 필수
- 비밀번호/API Key/PII는 코드에 절대 포함 금지

Approved Tools

- 사내 패키지 저장소: <https://npm.company.internal>
- 사내 Docker registry: registry.company.internal
- 사내 MCP 서버: mcp://internal.company.local

Restrictions

- 외부 패키지 신규 도입은 보안팀 승인 후

Org-wide

조직 전체 적용

Policy

준수 정책

IT 배포

MDM/Ansible

Read-only

사용자 수정 불가

/init 명령으로 자동 생성

CLAUDE.md 초안 자동 생성

```
$ cd /path/to/your/project
$ claude
> /init
```

Claude가 프로젝트를 분석 중...

- ✓ package.json 분석 (TypeScript, React, Vite 감지)
- ✓ tsconfig.json 확인 (strict 모드)
- ✓ scripts 분석 (dev/build/test/lint)
- ✓ 디렉토리 구조 스캔 (src/, tests/, docs/)
- ✓ README.md 분석
- ✓ 주요 코드 패턴 분석 (Functional Components, Hooks)

다음 ./CLAUDE.md 초안을 작성했습니다:

```
# Project Overview
...
```

이 내용을 저장하시겠습니까? [Y/n]

/init

/init

슬래시 명령

Auto-scan

프로젝트 자동 분석

Confirm

저장 전 확인

Editable

직접 편집 가능

/memory 편집 명령

대화형 메모리 편집

> /memory

/memory

Memory Files:

1. Project ./CLAUDE.md [edit]
2. User ~/.claude/CLAUDE.md [edit]
3. Local ~/.claude/local.md [edit]
4. Enterprise /etc/claude/CLAUDE.md [read-only]

Select [1-4]: 1

```
# Project: my-app
## Stack
- TypeScript 5.4 / React 18 / Vite 5
...
```

```
[ 자동 편집기 열림 - $EDITOR 환경변수 기준 ]
# 편집 완료 후 :wq 또는 Ctrl-S
```

✓ ./CLAUDE.md 갱신됨 (다음 메시지부터 반영)

/memory
메모리 편집

\$EDITOR
시스템 편집기

Immediate
다음 메시지부터

Enterprise
읽기 전용

단축 추가 문법

대화 중 메모리에 빠른 추가

shortcut

사용 예시

> 이 프로젝트는 pnpm 사용해. 항상 pnpm으로 명령 실행해 줘.
pnpm 사용 (npm/yarn 대신)

✓ ./CLAUDE.md > ## Tools 섹션에 추가됨
"pnpm 사용 (npm/yarn 대신)"

다른 예

> # 모든 PR은 반드시 reviewer 1명 이상 지정 필수
✓ ./CLAUDE.md > ## Workflow 섹션에 추가됨

어느 메모리에 추가할지 명시

> # @user 개인 들어쓰기는 2칸 선호
✓ ~/.claude/CLAUDE.md > ## Code Style 섹션에 추가됨

단축 문법: 메시지 시작 부분에

Claude가 적절한 섹션을 추론해 추가

#

단축 prefix

Auto-section

섹션 자동 추론

@user

User 메모리 지정

한 줄

한 줄씩 추가

Import 문법

메모리 파일 모듈화

@import

CLAUDE.md 안에 다른 마크다운 파일을 임포트

Project: my-app

@./docs/architecture.md

@./docs/coding-style.md

@./.claude/local.md

또는 사내 표준

@/etc/claude/team-rules.md

@~/ .claude/personal-style.md

Specific Project Notes

- 이 프로젝트만의 특수 사정...

효과:

- 큰 CLAUDE.md를 여러 파일로 분할 관리

- 사내 표준 파일을 여러 프로젝트가 공유

- 임포트는 재귀적으로 처리 (단, 순환 참조 방지)

@./path

임포트 prefix

Recursive

재귀 처리

No-cycle

순환 참조 방지

Modular

모듈화 가능

중첩 import와 우선순위

실제 병합 결과 추적

PRIORITY

적용 우선순위 예시

Enterprise /etc/claude/CLAUDE.md
 @/etc/claude/security-policy.md
 @/etc/claude/approved-tools.md

User ~/.claude/CLAUDE.md
 @~/.claude/code-style.md

Project ./CLAUDE.md
 @./docs/architecture.md
 @./docs/api-conventions.md

Local ~/.claude/local.md

충돌 시 (예: 들여쓰기 4칸 vs 2칸)

1. Local → 가장 강함 (개인 X 프로젝트)
2. Project → 팀 컨벤션
3. User → 개인 기본값
4. Enterprise → 조직 정책

Claude는 모든 내용을 합쳐 컨텍스트로 활용

Local

가장 우선

Project

팀 컨벤션

User

개인 기본값

Enterprise

조직 정책

CLAUDE.md 베스트 프랙티스 1

구조와 분량 가이드

BP 1

간결하게

CLAUDE.md는 매 세션 토큰을 소비합니다. 500-1500 단어가 적정선.

BP 2

의미 있는 섹션

Stack, Commands, Conventions 같은 명확한 섹션 헤더 사용.

BP 3

실행 가능 정보

"테스트 어떻게" 보다 "pnpm test" 같은 정확한 명령을 적기.

BP 4

의도적 갱신

결정사항 바뀌면 즉시 # 단축. 한 달에 한 번 정리.

WHY

CLAUDE.md는 코드 다음으로 자주 읽히는 문서가 되며, 정확하고 최신인 상태를 유지해야 Claude가 정확한 작업을 할 수 있음.

500-1500

권장 단어 수

Sections

명확한 구조

Executable

명령 그대로 실행

Living

살아 있는 문서

CLAUDE.md 베스트 프랙티스 (2)

내용 작성 가이드

BP 5

Do와 Don't

권장과 금지를 명확히. "X 사용", "Y 금지" 형식으로 강하게.

BP 6

예시 코드 포함

추상적 설명보다 실제 코드 스니펫이 훨씬 강력함.

BP 7

경로/명령 정확히

경로는 상대/절대 명확히, 명령은 그대로 복사 실행 가능하게.

BP 8

Off-limits 명시

건드리면 안 되는 영역(자동 생성, 레거시)을 명시.

EXAMPLE

"테스트 작성 시 React Testing Library만 사용. Enzyme 사용 금지." 처럼 강제 사항은 단호하게 적으세요. !!!

Do/Don't

명확한 방향

Snippets

코드 예시 효과적

Paths

경로 정확히

Off-limits

금지 영역 표기

CLAUDE.md 안티 패턴

피해야 할 작성 방식

안티 패턴	왜 나쁜가	대안
소설처럼 긴 설명	토큰 낭비, 핵심 흐려짐	짧은 항목 리스트
모호한 표현	"좀 더", "보통" 등	구체적 명령/숫자
오래된 정보 방치	잘못된 명령 실행 위험	정기 점검 (월 1회)
보안 비밀 포함	Git에 노출, 컨텍스트 유출	환경변수/시크릿 매니저
중복 정보 (Code/README와)	서로 다른 진실의 원천	CLAUDE.md → 다른 곳 참조
아무 의견 다 적기	정보 과부하	작업에 영향 주는 것만

Anti

피할 패턴

Verbose

장황함 = 비용

Secrets

비밀 포함 금지

Sync

코드와 동기 유지

실전 예시 / Python 프로젝트

Python/Django 프로젝트 CLAUDE.md

Python project example

```
# Project: order-service

## Stack
- Python 3.12 / Django 5.0 / DRF
- PostgreSQL 16 / Celery + Redis
- pytest + pytest-django

## Commands
- 개발: python manage.py runserver
- 마이그레이션: python manage.py migrate
- 새 마이그레이션: python manage.py makemigrations
- 테스트: pytest -x --cov=src
- 린트: ruff check . && mypy src/

## Conventions
- Type hints 필수 (모든 함수/메서드)
- async View 사용 금지 (sync 만)
- 트랜잭션: @transaction.atomic 데코레이터 활용
- 비즈니스 로직은 service 레이어로 분리

## Don't
- DB에 직접 SQL 실행 X (Django ORM 사용)
- requirements.txt 직접 수정 X (poetry add 사용)
```

Python
실전 예시 1

Django
프레임워크

Concrete
구체적 명령

Don't
명확한 금지

실전 예시 / 모노레포

여러 패키지 관리 CLAUDE.md

Monorepo example

```
# Monorepo: company-platform
```

```
## Structure (Turborepo)
```

- apps/web Next.js 15 프론트엔드
- apps/api Fastify 4 백엔드
- apps/admin Admin SPA
- packages/ui 공유 컴포넌트 라이브러리
- packages/types 공유 TypeScript 타입
- packages/utils 공통 유틸 함수

```
## Commands (루트에서)
```

- 전체 개발: pnpm dev
- 특정 앱만: pnpm dev --filter web
- 빌드: pnpm build
- 패키지 전체 테스트: pnpm test
- 한 패키지만 테스트: pnpm test --filter @company/ui

```
## Conventions
```

- 새 공유 코드는 packages/ 아래 (apps 간 직접 import 금지)
- 버전 관리: changesets (pnpm changeset)
- 의존성 추가: pnpm add -F <package-name>

```
@./docs/architecture.md
```

```
@./docs/release-process.md
```

Monorepo

실전 예시 2

Turbo

pnpm + Turborepo

--filter

특정 패키지 지정

@import

문서 분할 활용

CHAPTER 01 - Part 09

Overview – Workflow Patterns

Claude Code의 정체성, 아키텍처, 설치, 인증, 핵심 기능, 워크플로 패턴까지



Part 9 / Workflow Patterns

Claude Code 실무 활용 패턴

PART OBJECTIVE

단순 도구 사용을 넘어 실무에서 효과적으로 활용하는 8가지 워크플로 패턴과 비용/컨텍스트 관리, 안티 패턴까지 학습합니다.

PATTERN

8가지 워크플로

EPC, TDD, 리뷰, 멀티 에이전트, Visual, Headless, Pipeline, CI/CD

COST

비용 관리

/cost 명령으로 토큰 사용량 모니터링과 최적화

CONTEXT

컨텍스트 관리

/clear vs /compact 선택, 효과적 토큰 사용 전략

ANTI

안티 패턴

실무에서 흔히 마주치는 잘못된 사용 패턴과 대안

18 Slides

Part 9 분량

~35 min

예상 강의 시간

8 Patterns

워크플로 패턴

Lab 4-5

실습 포함

Pattern 1 / Explore-Plan-Code

EPC 3단계 워크플로

STEP 01

Explore

탐색만, 수정 X

STEP 02

Plan

계획 제시 + 합의

STEP 03

Code

실행 + 검증

WHY EPC

큰 작업에 바로 코드 수정을 시작하면 잘못된 방향으로 갈 위험이 큼. 탐색-계획-실행 3단계로 위험을 통제.

EXPLORE

읽기만

Read, Grep, Glob으로 코드베이스 이해.
절대 수정 X.

PLAN

명시적 합의

"무엇을 어떻게" 텍스트로 정리.
사용자가 동의해야 진행.

CODE

계획대로 실행

합의된 범위 내에서만 Edit/Write/Bash.
벗어나면 다시 Plan.

Explore

읽기 전용 1단계

Plan

합의 후 진행

Code

계획 안에서만

Loop

벗어나면 재계획

EPC 패턴 실전 예시

결제 모듈에 retry 추가

EPC EXAMPLE

> 결제 모듈에 자동 재시도 로직을 추가해 주세요. (Plan 모드)

[Explore]

- ✓ Read src/payment/processor.ts
- ✓ Read src/payment/errors.ts
- ✓ Grep "PaymentError" → 5 callsites
- ✓ Read tests/payment/processor.test.ts

[Plan]

1. exponential backoff 적용 (1s, 2s, 4s)
2. 재시도 가능 에러 분류: NetworkError, TimeoutError
3. 멍등성 검증: paymentIntentId로 중복 방지
4. 최대 재시도 3회 (이후 dead letter queue)
5. 새 테스트 케이스 4개 추가

- ✓ 위 계획으로 진행하시겠습니까?

> Yes

[Code]

- ✓ Edit src/payment/processor.ts
- ✓ Write src/payment/retry.ts
- ✓ Edit tests/payment/processor.test.ts
- ✓ Bash npm test → all green

Explore

4 reads + 1 grep

Plan

5 step plan

Confirm

명시적 승인

Code

계획대로 실행

Pattern 2 / TDD Workflow

Test-First 개발 사이클

STEP 01

Red

실패 테스트 작성

STEP 02

Green

최소 구현

STEP 03

Refactor

구조 개선

STEP 04

Repeat

다음 테스트

WHY TDD WITH CLAUDE

TDD는 Claude의 능력과 잘 맞습니다. 실패하는 테스트가 명확한 목표가 되어 Claude가 헤매지 않고 빠르게 작업할 수 있습니다.

> userService.changePassword 에 대한 TDD를 시작합니다.

TDD example

Claude:

1. Write tests/userService.changePassword.test.ts (실패 테스트 5개)
2. Bash npm test → 5 failures
3. Edit src/services/userService.ts (최소 구현)
4. Bash npm test → all green
5. Edit src/services/userService.ts (Refactor)
6. Bash npm test → all green

Red

실패부터 시작

Green

최소 구현

Refactor

안전한 정리

Fast

Claude와 빠른 사이클

Pattern 3 / Code Review

PR 자동 리뷰 + 가이드

CODE REVIEW

```
# 1. 본인 PR 셀프 리뷰
$ claude
> diff main..HEAD 한 후 발견 가능한 잠재적 이슈를 알려 줘
$ claude

Claude:
✓ Bash git diff main..HEAD --stat
✓ Bash git diff main..HEAD -- 'src/'
✓ Review 출력:
  - src/api/users.ts:142 N+1 query 위험 (loop in Promise.all)
  - src/utls/format.ts:67 toFixed로 precision 손실 가능
  - test 커버리지: 신규 코드 78% (목표 85% 미달)

# 2. 다른 사람 PR 리뷰
$ gh pr checkout 123
$ claude -p "이 PR이 우리 CLAUDE.md 컨벤션을 따르는지 검토해 줘"

# 3. CI 통합 (헤드리스)
$ claude -p "$(git diff main..HEAD)" --output-format json \
  > review.json
```

Self-review

PR 셀프 리뷰

Convention

CLAUDE.md 기준

CI

CI 자동 통합

JSON

구조화 출력

Pattern 4 / Multi-Agent

Subagents 병렬 처리 (Chapter 2 맛보기)

SUBAGENTS

> 대규모 마이그레이션 작업을 Subagents로 분할해 주세요.

Main Claude:

- ✓ 작업 분석: 47개 파일을 5개 영역으로 분할
- ✓ Subagent 1: src/api/* (10 files) - api-migrator
- ✓ Subagent 2: src/services/* (12 files) - service-migrator
- ✓ Subagent 3: src/components/* (15 files) - ui-migrator
- ✓ Subagent 4: tests/* (8 files) - test-migrator
- ✓ Subagent 5: docs/* (2 files) - doc-updater

병렬 실행:

- Subagent 1-4 동시 진행 (각각 격리된 컨텍스트)
- Subagent 5는 1-4 완료 후 실행 (의존성)

- ✓ 모든 결과 취합 후 통합 PR 생성

Chapter 2에서 Subagents 시스템을 깊이 다룹니다.

5 Subagents

병렬 처리

Parallel

동시 실행

Isolated

컨텍스트 격리

Aggregate

결과 취합

Pattern 5 / Visual Workflow

스크린샷과 디자인 기반 작업

VISUAL

```
# 1. 디자인 시안 → 코드
$ claude
> [이미지 첨부: figma-design.png]
> 이 디자인을 React 컴포넌트로 구현해 주세요.
```

```
Claude:
✓ Read figma-design.png (이미지 분석)
✓ 색상, 레이아웃, 타이포그래피 추출
✓ Write src/components/PricingCard.tsx
✓ Write src/components/PricingCard.module.css
```

```
# 2. 버그 스크린샷 → 수정
$ claude
> [이미지 첨부: bug-screenshot.png]
> 이 UI에서 버튼이 잘려 보입니다. 원인을 찾고 수정해 주세요.
```

```
Claude:
✓ Read bug-screenshot.png
✓ 잘린 영역 식별 (overflow 추정)
✓ Grep 관련 컴포넌트 검색 → Read → Edit
```

Image

시안/스크린샷 첨부

Multi-modal

Claude 멀티모달

Design→Code

시안 구현

Bug→Fix

시각 버그 추적

Visual Workflow 활용 예시

UI/UX 작업 시나리오

SCENARIO 1

디자인 → 코드

Figma 시안을 첨부해 React/Vue 컴포넌트로 구현. CSS-in-JS 또는 Tailwind 자동 선택.

SCENARIO 2

A/B 비교

현재 화면과 목표 화면 스크린샷을 함께 첨부해 변경점 자동 식별.

SCENARIO 3

반응형 점검

여러 viewport 스크린샷을 첨부해 모바일/태블릿/데스크톱 호환성 점검.

TIP

PNG/JPEG/WEBP 형식이 가장 안정적입니다. PDF도 가능하지만 페이지가 많으면 토큰 비용이 큼니다.

PNG/JPG

권장 포맷

<2MB

권장 파일 크기

Figma

디자인 도구 연동

Crop

핵심 영역만 첨부

Pattern 6 / Headless Automation

-p 플래그로 비대화 자동화

HEADLESS

```
# 기본 헤드리스 사용
$ claude -p "package.json에서 outdated 패키지 알려줘"

# 결과를 파일로 저장
$ claude -p "이 프로젝트의 의존성 분석" > deps-report.md

# 다른 명령과 파이프
$ git log --oneline -20 | claude -p "지난 20개 커밋의 영향도 요약"

# JSON 출력으로 후처리
$ claude -p "사용된 환경변수 목록" --output-format json | jq '.envvars'

# 도구 제한 + 비밀번호 없는 자동화
$ claude -p "코드베이스의 보안 이슈 스캔" \
  --allowed-tools "Read,Grep,Glob" \
  --output-format json

# 모델 지정으로 비용 절감
$ claude -p "간단한 변환 작업" --model haiku
```

-p
Headless 플래그

Pipe
stdin/stdout 통합

JSON
구조화 후처리

--model
모델 지정 비용 절감

Pattern 7 / Pipeline

stdin/stdout 기반 데이터 처리

PIPELINE

```
# 1. 단순 파이프
$ git log --since="1 week ago" --pretty=format:"%h %s" \
  | claude -p "지난 주 변경 사항을 카테고리별로 분류해 줘"

# 2. 여러 단계 파이프
$ find . -name "*.test.ts" \
  | claude -p "이 테스트 파일 목록에서 누락된 도메인 찾기" \
  | grep -i "missing"

# 3. JSON 파이프 (구조화 데이터)
$ aws ec2 describe-instances --output json \
  | claude -p "비용 최적화 제안" --output-format json \
  | tee optimization-report.json

# 4. 로그 분석 자동화
$ tail -1000 /var/log/app.log \
  | claude -p "이 로그에서 ERROR 패턴과 빈도를 정리해" \
  > daily-error-report.md
```

stdin

데이터 입력

stdout

결과 출력

Multi-stage

여러 단계 파이프

JSON

구조화 데이터

JSON 출력 활용

구조화된 결과로 후처리

JSON OUTPUT

```
# JSON 출력 옵션
$ claude -p "현재 디렉토리 분석" --output-format json

{
  "model": "claude-sonnet-4-5",
  "tokens": { "input": 1240, "output": 670, "cost_usd": 0.012 },
  "summary": "...",
  "tools_used": [
    { "tool": "Read", "count": 5 },
    { "tool": "Grep", "count": 2 }
  ],
  "files_modified": [],
  "files_read": [
    "package.json", "src/index.ts", "README.md"
  ],
  "structured_output": {
    "stack": ["typescript", "express"],
    "issues": [
      { "severity": "high", "file": "src/auth.ts:42", ... }
    ]
  }
}

# jq로 추출
$ claude -p "..." --output-format json | jq '.structured_output.issues'
```

JSON
머신 친화 포맷

jq
필드 추출

tokens
비용 추적

tools_used
도구 호출 추적

Pattern 8 / CI/CD Integration

GitHub Actions 자동화

```
# .github/workflows/claude-review.yml
name: Claude Code Review
on:
  pull_request:
    branches: [main]
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }

      - name: Install Claude Code
        run: npm install -g @anthropic-ai/claude-code

      - name: Run Review
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
          CLAUDE_DISABLE_UPDATE_CHECK: '1'
        run: |
          git diff origin/main..HEAD \
          | claude -p "이 PR을 CLAUDE.md 기준으로 리뷰해 줘" \
            --allowed-tools "Read,Grep,Glob" \
            --output-format json \
            > review.json

      - name: Comment on PR
        run: gh pr comment --body-file review.json
```

CI/CD

Actions

GitHub 워크플로

secrets

API Key 안전

allow-list

읽기 전용 안전성

PR comment

결과 자동 게시

비용 관리 / /cost

토큰 사용량 모니터링

> /cost

/cost

Current Session

Model	Input tokens	Output tokens	Cost (USD)
sonnet-4-5	45,231 (cached:8,420)	12,453	\$0.328
haiku-4-5	2,140 (subagent)	892	\$0.013

Tools used 38 calls (Read:18, Edit:8, Bash:7, Grep:5)

Today (UTC)

Total cost: \$1.42
Sessions: 7
Most expensive: 'Refactoring auth module' (\$0.92)

Daily limit: \$20.00 (\$1.42 / \$20.00 = 7.1%)

To save costs:

- 큰 작업 분할
- 단순 작업은 --model haiku
- 컨텍스트는 /clear로 정기 초기화

/cost

명령으로 즉시 확인

Daily

일일 사용량 누적

Cached

캐시 토큰 별도

Limit

한도 설정 가능

컨텍스트 관리 / /clear vs /compact

두 명령의 적절한 선택

/CLEAR

컨텍스트 완전 초기화

모든 메시지를 지움 (CLAUDE.md는 재로드)
토큰 사용량 0으로 리셋
작업 전환 시 (다른 기능)
컨텍스트 오염 의심 시
가장 안전한 선택

/COMPACT

오래된 메시지 요약

핵심 정보 유지하며 토큰 감소
50-70% 토큰 절감 효과
같은 작업 흐름을 계속 이어갈 때
디테일 손실 발생 가능 (lossy)
정밀 작업에는 부적합

/clear

완전 초기화

/compact

요약 압축

70%

compact 자동 안내 임계

Strategy

작업 성격에 맞춰

토큰 효율 베스트 프랙티스

비용과 응답 속도 모두 개선

BP 1

범위 제한

"전체 코드베이스" 대신
"src/api/*"처럼 영역을 명시.

BP 2

단계 분할

큰 작업을 여러 세션으로 분할.
각 세션 종료 시 /clear.

BP 3

모델 선택

단순 변환은 haiku, 복잡 추론
은 opus, 기본은 sonnet.

BP 4

CLAUDE.md 활용

반복 설명을 CLAUDE.md에
영구 저장해 매 세션 시간 절약.

IMPACT

베스트 프랙티스 적용 시 토큰 사용량 30-50% 감소, 응답 속도 1.5-2x 개선 효과를 기대할 수 있습니다.

30-50%

토큰 절감 효과

1.5-2x

속도 개선

Practice

꾸준한 적용

Measure

/cost로 효과 측정

자주 마주치는 안티 패턴

실무에서 피해야 할 사용 방식

안티 패턴	문제점	대안
"전부 다 해줘"	범위 모호 → Claude가 헤맴	명확한 단계 분할
CLAUDE.md 없이 시작	매번 같은 컨텍스트 재설명	/init으로 즉시 작성
컨텍스트 가득찰 때까지 사용	응답 품질 저하	70%에서 /compact 또는 /clear
Plan 모드 안 쓰고 큰 변경	잘못된 방향 위험	Shift+Tab으로 Plan 우선
Yolo 모드 일상 사용	돌이킬 수 없는 실수 위험	allow/ask 권한 모델
모든 도구를 allow	위험 명령 실행 가능	필요한 것만 allow

Vague

모호한 지시 X

Plan

큰 작업은 항상 Plan

Permission

권한은 최소로

Manage

컨텍스트 능동 관리

패턴 조합 예시

실무에서 여러 패턴 결합

COMBINED PATTERNS

```
# 예시: 신규 기능 + CI 자동 리뷰 + 비용 통제

# 1. EPC + Plan 모드로 신규 기능 개발
$ claude
> Shift+Tab (Plan 모드 활성화)
> /users API에 페이지네이션 추가

# 2. 큰 변경은 여러 Subagents 사용 (Pattern 4)
> 이 변경을 routes/services/tests 3개 Subagent로 병렬 처리

# 3. 비용 점검 후 다음 작업 결정 (Pattern 비용관리)
> /cost
# 사용량 확인 후 다음 작업 진행 여부 결정

# 4. PR 생성 → CI 자동 리뷰 (Pattern 8)
> git checkout -b feat/users-pagination && commit && gh pr create
# GitHub Actions가 자동으로 Claude로 리뷰 댓글 게시

# 5. 컨텍스트 정리 후 다음 작업
> /clear
> 다음 기능: ...
```

EPC + Plan

안전한 개발

Multi-agent

병렬 가속

Cost

비용 점검

CI

자동 리뷰

Part 9 Recap

Workflow Patterns 핵심 정리

TAKEAWAY 1

8가지 패턴

EPC, TDD, Review, Multi-Agent, Visual, Headless, Pipeline, CI/CD

TAKEAWAY 2

컨텍스트 관리

/clear와 /compact를 작업 성격에 맞게 적절히 선택

TAKEAWAY 3

비용 의식

/cost로 모니터링하며 모델 선택과 범위 제한으로 절감

TAKEAWAY 4

안티 패턴 회피

모호 지시 X, Plan 우선, 권한 최소화, 능동 컨텍스트 관리

NEXT - PART 10

Part 10은 Chapter 1 마무리. 핵심 요약, FAQ, 5가지 실습 랩 가이드, 다음 챕터 예고를 14장에 걸쳐 다룹니다.

Part 9

완료

Part 10

Recap & Labs (14p)

5 Labs

챕터 1 실습

~30 min

잔여 시간

CHAPTER 01

Overview – Summary

Claude Code의 정체성, 아키텍처, 설치, 인증, 핵심 기능, 워크플로 패턴까지



Chapter 1 핵심 요약

9개 Part 199장의 압축본

IDENTITY

Agentic Coding

CLI 기반 자율 실행,
Anthropic 1차 통합,
Read/Edit/Run 3원 능력

ARCHITECTURE

4계층 설계

Client, Core, Models,
Tools, 그리고 다층 보안
Defense in Depth

OPERATIONS

설치/인증/도구

npm/Native/Docker, 4가
지 Auth, 11+ 내장 도구,
MCP 확장

WORKFLOW

8가지 패턴

EPC, TDD, Review,
Multi-Agent, Visual,
Headless, Pipeline,
CI/CD

OUTCOME

여러분은 이제 Claude Code의 정체성, 내부 구조, 설치 및 운영, 그리고 일상 워크플로 패턴까지 모두 이해했습니다.
다음 챕터부터 Subagents, Settings, SDK로 한 단계씩 심화합니다.

9 Parts

학습 완료

186 Slides

Chapter 1 분량

~3.5h

강의 시간

Next

Chapter 2 Agents

학습 목표 달성 체크리스트

본인의 학습을 점검하세요

Objective	확인 질문	관련 Part
Define	Copilot/Cursor와의 차이를 설명할 수 있는가	Part 1
Architecture	Tool Router 5단계 흐름을 설명할 수 있는가	Part 2
Install	본인 환경에 설치하고 doctor를 통과했는가	Part 3
Authenticate	4가지 인증 옵션 중 적절한 것을 선택했는가	Part 4
Use Tools	Read/Edit/Bash/Grep을 직접 사용해 봤는가	Part 5, 6
Context & Memory	CLAUDE.md를 작성하고 /clear를 활용했는가	Part 8
Workflow Patterns	EPC 또는 Plan 모드로 작업을 완수했는가	Part 9

7

학습 목표

Self-Check

자가 점검

Lab

실습으로 확인

Mentor

미달성 시 코치 요청

자주 묻는 질문 (FAQ)

Chapter 1 관련 빈출 Q&A

FAQ

Q1. Claude Code와 Claude.ai 챗은 무엇이 다른가요?

A. Claude.ai는 일반 챗 인터페이스, Claude Code는 코딩 전용 에이전트. 도구 호출, 파일 시스템 접근, 권한 모델 등 코딩 워크플로 최적화.

Q2. 사내망에서 사용하려면 어떻게 해야 하나요?

A. Bedrock 또는 Vertex AI 경로 + 사내 프록시 설정이 일반적. 상세는 Chapter 3 Admin Setup에서 다룸.

Q3. 비용이 부담스럽습니다. 어떻게 절감하나요?

A. (1) Sonnet 기본 사용 (2) /cost 모니터링 (3) 작업 범위 축소 (4) /clear 적절 활용 (5) Subagent로 작은 모델 위임

Q4. 코드가 학습에 사용되나요?

A. 기본적으로 학습에 사용되지 않음. ZDR 옵션도 제공.

Q5. 다른 LLM도 사용 가능한가요?

A. Claude Code는 Claude 모델 전용. 단, Bedrock/Vertex 경로로 동일 모델을 다른 청구 경로로 호출 가능.

5+ Q

대표 질문

Doc

공식 문서 참조

Slack

사내 채널 활용

Support

Anthropic Support

추가 학습 자료

Chapter 1 보충 및 심화 자원

카테고리	자원	활용
공식 문서	https://code.claude.com/docs	레퍼런스용
GitHub	github.com/anthropics/claude-code	이슈/예제 참고
AWS Bedrock	aws.amazon.com/bedrock/claude	엔터프라이즈 통합
Vertex AI	cloud.google.com/vertex-ai	GCP 통합
MCP 생태계	modelcontextprotocol.io	도구 확장 학습
커뮤니티	discord.gg/anthropic , Reddit r/ClaudeAI	실시간 토론

Docs

공식 레퍼런스

GitHub

예제 코드

Discord

커뮤니티 토론

Blog

Anthropic 블로그

Lab 1 / 설치 및 첫 세션

15분 실습

STEP 01

환경 점검
(node, git, IDE)

STEP 02

설치
(npm 또는 native)

STEP 03

claude doctor
실행

STEP 04

/login 또는
API key 설정

STEP 05

첫 프롬프트
응답 확인

```
# 단계별 명령
$ node --version           # v18+ 확인
$ npm install -g @anthropic-ai/claude-code
$ claude --version
$ claude doctor           # 모든 ✓ 확인
$ cd ~/your-test-project
$ claude
> "이 프로젝트의 구조를 간단히 설명해 주세요"
```

```
# 성공 기준
✓ doctor 통과   ✓ 인증 완료   ✓ 첫 응답 수신   ✓ Read 도구 호출 관찰
```

Lab 1

15 min

예상 소요

doctor

검증 기준

Auth

4 옵션 중 1

First call

첫 응답

Lab 2 / CLAUDE.md 작성

20분 실습

Lab 2

```
# 1. 본인 프로젝트 루트에서
$ claude
> /init                                # 자동 생성 시도

# 2. 생성된 CLAUDE.md를 본인 프로젝트에 맞춰 편집
# 예시 구조:
# ## Project Overview      - 한국어 1-2 문장 (목적/사용자)
# ## Tech Stack            - Node 20 LTS, Express, PostgreSQL
# ## Commands             - 빌드/테스트/런트 명령
# ## Conventions          - 함수형, ES Modules, 에러 처리
# ## Architecture         - 폴더 구조와 흐름

# 3. 다시 세션 시작 후 동작 확인
$ claude
> 새 endpoint /healthz를 추가해 주세요
```

20 min

예상 소요

/init

자동 생성 명령

5 sections

핵심 섹션

Apply

재세션으로 적용

Lab 3 / 디버깅 시나리오

20분 실습

Lab 3

```
# 1. 의도적으로 버그가 있는 코드 준비
# 또는 본인 프로젝트의 알려진 이슈 활용

# 2. Claude에 에러 메시지를 그대로 전달
$ Claude
> 다음 에러가 발생합니다. 원인을 찾고 수정해 주세요.
>
> TypeError: Cannot read property 'name' of undefined
>   at UserController.getUser (src/controllers/user.js:42)

# 3. Claude의 자율 작업 관찰
#   - Read 도구로 user.js 읽기
#   - Grep으로 getUser 호출지 검색
#   - Edit으로 null 체크 추가
#   - Bash로 npm test 실행

# 4. 결과 확인
#   - 변경된 파일 검토
#   - 테스트 통과 확인
#   - 추가 권장사항 검토
```

20 min

예상 소요

Real bug

실제 에러 활용

Observe

도구 호출 관찰

Verify

테스트로 검증

Lab 4 / Plan 모드 활용

20분 실습

Lab 4

```
# 1. claude 시작 후 Plan 모드 진입
$ claude
> [Shift+Tab]           # Plan 모드 토글
# 또는 처음부터:
$ claude --plan

# 2. 복잡한 작업을 요청
> 사용자 모델에 soft delete 기능을 추가해 주세요.
> deleted_at 컬럼, 미들웨어, 마이그레이션, 테스트를 모두 포함해야 합니다.

# 3. Plan 검토
#   - Claude가 단계별 계획 제시
#   - 영향 받는 파일 목록
#   - 변경 요약과 위험 분석

# 4. 승인 또는 수정
> 좋은데, 마이그레이션은 분리해 별도 PR로 만들어 주세요.

# 5. 실행 결과 검토 및 Git 커밋
```

20 min

예상 소요

Shift+Tab

Plan 모드 진입

Review

계획 검토

Iterate

수정 반복

Lab 5 / Headless 자동화

25분 실습

Lab 5

```
# 1. 간단한 헤드리스 호출
$ claude -p "최근 git log 5개를 요약해 주세요" --output-format text

# 2. JSON 출력으로 후처리
$ claude -p "package.json 의존성 분석" --output-format json | jq '.summary'

# 3. 셸 파이프 통합
$ cat error.log | claude -p "이 에러 로그를 분류해 주세요"

# 4. 자동화 스크립트 예시
$ cat > review.sh << 'EOF'
#!/bin/bash
DIFF=$(git diff HEAD~1)
echo "$DIFF" | claude -p "이 PR diff를 검토해 주세요" \
  --allowed-tools "" --output-format text > review.md
EOF

# 5. cron 또는 GitHub Actions로 등록 (Chapter 5에서 심화)
```

25 min

예상 소요

-p

Headless 플래그

JSON

구조화 출력

pipe

셸 통합

Q & A

질문과 토론 시간 (15분)

OPEN DISCUSSION

Chapter 1 학습 내용에 대해 자유롭게 질문하고 토론하는 시간입니다.
본인 환경에서 발생한 문제, 적용 사례, 도입 시 우려사항 등 무엇이든 환영합니다.

INSTALL

설치/인증 문제

본인 환경에서 만난 설치 또는 인증 문제를 공유해 주세요.

WORKFLOW

도입 시나리오

본인 팀에 적용할 때 우려되는 사항이나 적합한 시나리오를 토론합니다.

POLICY

보안/정책

사내 보안 정책과 충돌하는 부분, 데이터 거버넌스 질문을 다룹니다.

15 min
Q&A 시간

Open
자유 토론

Share
경험 공유

Mentor
코치 지원

Next Chapter Preview

Chapter 2 / Agents (Subagents) 예고

CHAPTER 2 OVERVIEW

Chapter 2에서는 Subagents 시스템을 다룹니다. 작업을 분담하고 병렬화하여 더 큰 작업을 처리할 수 있게 합니다.

TOPIC 1

Subagent 개념

메인 에이전트에서 자식 에이전트를 위임하는 메커니즘 이해

TOPIC 2

맞춤 에이전트 정의

.claude/agents/ 디렉토리에 맞춤 서브에이전트 작성

TOPIC 3

Task 도구

Task 도구로 서브에이전트를 자동 디스패치하는 패턴

TOPIC 4

실전 예제

코드 리뷰어, 테스터, 보안 검사기 등 5가지 실전 패턴

Day 1 PM

진행 시간

110 Slides

Chapter 2 분량

5 Patterns

실전 패턴

Lab

맞춤 에이전트 작성

References

참고 자료 및 출처

REFERENCES

Official Documentation

<https://code.claude.com/docs/ko/overview>
<https://code.claude.com/docs/ko/agents>
<https://code.claude.com/docs/ko/admin-setup>
<https://code.claude.com/docs/ko/settings>
<https://code.claude.com/docs/ko/cli-reference>
<https://code.claude.com/docs/ko/agent-sdk/overview>

GitHub

<https://github.com/anthropics/claude-code>
<https://github.com/anthropics/anthropic-cookbook>

Cloud Integrations

AWS Bedrock: <https://aws.amazon.com/bedrock/claude/>

Ecosystem

Model Context Protocol: <https://modelcontextprotocol.io>
Anthropic Discord: discord.gg/anthropic

Docs

공식 문서 (한국어)

GitHub

소스/예제

Cloud

AWS

MCP

생태계 표준

Thank you!

Choi, WooHyung / Prin. Solutions Architect / AWS Korea

whchoi@amazon.com

<https://linkedin.com/in/woohyungchoi>

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

